

Tutorial code: 30-day unplanned readmission after an ICU stay (MIMIC-IV v3.1)

Contents: the complete tutorial notebook in Python, including all comments and explanatory text. Each shaded block is one code cell; entering the cells in order in a Jupyter notebook and running them reproduces the analysis.

Outputs: cleared. MIMIC-IV is credentialed data, so results are produced by running the code on a local copy of MIMIC-IV v3.1 obtained through PhysioNet.

Requirements: Python 3.10 or newer with pandas, NumPy, scikit-learn, matplotlib and joblib (pyarrow optional).

Location: https://github.com/AdachiHouse45/Al-for-Healthcare/blob/main/mimic_readmission_tutorial_code.pdf

Predicting 30-Day Unplanned Readmission After an ICU Stay

A self-learning tutorial on MIMIC-IV v3.1

Course: AI 380H — AI in Healthcare

Format: step-by-step tutorial notebook. Every cell is commented so that a reader can reproduce the project from scratch.

What this tutorial teaches

By the end of this notebook you will be able to:

1. **Frame** a clinical prediction problem precisely (who, when, what outcome, what horizon).
2. **Build a cohort** from raw MIMIC-IV tables, with an auditable inclusion/exclusion flow.
3. **Engineer features without temporal leakage**, that is, without information that would not be available at prediction time, a common source of error in clinical prediction models.
4. **Split data by patient**, not by row, so the same person cannot appear in train and test.
5. **Train and compare four model families:** penalised logistic regression (the baseline), random forest, gradient boosting, and a small feed-forward neural network.
6. **Evaluate for an imbalanced clinical problem:** AUROC alone is insufficient, so we also use AUPRC, calibration, Brier score, and a decision-relevant operating point (alert rate / PPV / number-needed-to-evaluate).
7. **Explain** the model with coefficients, permutation importance, and partial dependence.
8. **Audit** performance across patient subgroups and interpret the differences.
9. **Validate over time** with a temporal split (train on earlier years, test on later ones).

The clinical question

For an adult who survived a hospitalisation that included an ICU stay, what is the probability of an **unplanned readmission to the hospital within 30 days of discharge**?

Thirty-day readmission is a standard quality and payment metric (in the U.S., CMS's Hospital Readmissions Reduction Program penalises hospitals with excess readmissions). Patients who have been critically ill are a high-risk group: they leave with new medications, deconditioning, and unresolved problems. A model that identifies high-risk patients at discharge allows a care team to direct limited transitional-care resources to the patients most likely to benefit.

Prediction time is the moment of hospital discharge. Every model input must be available at or before that moment; no information recorded after discharge is used.

Comparison with introductory MIMIC tutorials

Many introductory MIMIC tutorials predict **in-hospital mortality** from the first 24 hours of ICU vital signs. This project differs in the following respects:

	Typical introductory MIMIC tutorial	This tutorial
Outcome	In-hospital death	Post-discharge readmission (label is outside the index encounter)
Anchor	ICU admission	Hospital discharge
Tables	<code>icustays</code> , <code>chartevents</code>	<code>admissions</code> , <code>patients</code> , <code>icustays</code> , <code>diagnoses_icd</code> , <code>labevents</code> , <code>prescriptions</code>
Modelling focus	One model, AUROC	Four model families, calibration, subgroup audit, temporal shift
Extra skill taught	—	Charlson comorbidity coding from ICD-9 and ICD-10, group-aware splitting

Before you run anything

- **Point the notebook at your data.** Cell 1.2 looks for your MIMIC-IV folder in a `MIMIC_DIR` environment variable, then in a one-line `mimic_path.txt` file next to this notebook, then in a placeholder that can be edited. The first two options keep the local folder path out of the notebook file.
- **Data use.** MIMIC-IV is credentialed data under a PhysioNet Data Use Agreement. Do not upload the data or any row-level output to a shared or public location, and do not enter patient records into online AI services. Aggregate results (counts, metrics, plots) may be shared. This notebook does not print individual patient records (table *schemas* are shown instead of `.head()`), and all row-level files it writes are stored in `readmission_project/cache/`. Clear all outputs before sharing the notebook.
- **Runtime.** The core pipeline (small tables only) runs in roughly 2–5 minutes on a laptop. The two optional enrichment steps stream very large files and can take 10–40 minutes the first time; both are cached to disk afterwards.
- **Reproducibility.** After any edit, run *Kernel → Restart & Run All* to confirm that the notebook executes from top to bottom without errors.

Packages

Required: `pandas`, `numpy`, `scikit-learn`, `matplotlib`, `joblib`. Optional (the notebook runs without them): `pyarrow` (fast caching), `xgboost` (an extra model), `shap` (per-patient explanations).

```
pip install pandas numpy scikit-learn matplotlib joblib pyarrow jupyterlab
```

Tested with Python 3.12–3.13, pandas 2.3.3 / scikit-learn 1.7.2, and pandas 3.0.2 / scikit-learn 1.8.0.

1. Setup and configuration

Everything you might want to change lives in one cell (1.2). A reader should be able to point `MIMIC_DIR` at their own copy of MIMIC-IV (environment variable, `mimic_path.txt`, or the placeholder) and run the whole notebook unchanged.

```
# -----
# 1.1 Imports
# -----
import os
import sys
import json
import time
import gzip
import warnings
from pathlib import Path
from datetime import timedelta

import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt

import sklearn

# Keep the output readable: only expected, non-actionable warnings are silenced
# (e.g. sklearn telling us a rare category was unseen at transform time).
warnings.filterwarnings("ignore", category=FutureWarning)

# Any warning that does appear is printed WITHOUT the file path it came from. By default Python
# prints the full path of the library file (e.g. C:\Users\<your name>\...\site-packages\...),
# and everything printed is saved inside the .ipynb - so a stray warning would put your user
# name into any copy of the notebook you share.
def _warning_without_path(message, category, filename, lineno, line=None):
    return f"{category.__name__}: {message}\n"

warnings.formatwarning = _warning_without_path

pd.set_option("display.max_columns", 80)
pd.set_option("display.width", 160)

print("python      :", sys.version.split()[0])
print("pandas      :", pd.__version__)
print("numpy        :", np.__version__)
print("scikit-learn:", sklearn.__version__)
print("matplotlib   :", matplotlib.__version__)

# This notebook uses `get_feature_names_out`, `StratifiedGroupKFold` and
```

```

# `HistGradientBoostingClassifier`, which need scikit-learn 1.1 or newer.
_skl = tuple(int(x) for x in sklearn.__version__.split(".")[2:])
if _skl < (1, 1):
    print("\n[!] scikit-learn < 1.1 detected. Please run: pip install -U scikit-learn")

# -----
# 1.2 Configuration
# -----
# --- Location of the MIMIC-IV data -----
# Point this at the folder that directly contains the `hosp/` and `icu/` subfolders.
# The notebook checks three places, in this order, and uses the first one it finds:
# 1. an environment variable named MIMIC_DIR
# 2. a one-line text file named `mimic_path.txt`, next to this notebook, holding the path
# 3. MIMIC_DIR_PLACEHOLDER below, edited by hand
# Options 1 and 2 keep the local folder path out of the notebook file. If you use version
# control, exclude mimic_path.txt from it (for example, with a .gitignore entry).
MIMIC_DIR_PLACEHOLDER = r"/path/to/mimic-iv-3.1" # e.g. r"D:\data\mimic-iv-3.1" or "~/data/mimic-iv-3.1"

def find_mimic_dir():
    """Return (path, where_it_came_from), using the lookup order described above."""
    env_value = os.environ.get("MIMIC_DIR", "").strip().strip('\'').strip('"')
    if env_value:
        return Path(env_value).expanduser(), "the MIMIC_DIR environment variable"
    path_file = Path("mimic_path.txt") # relative to the notebook's own folder
    if path_file.exists():
        # "utf-8-sig" swallows the invisible byte-order mark some Windows editors add, and
        # the strip() calls tolerate the quotes that File Explorer's "Copy as path" adds.
        text = path_file.read_text(encoding="utf-8-sig").strip().strip('\'').strip('"')
        if text:
            return Path(text).expanduser(), "mimic_path.txt"
    return Path(MIMIC_DIR_PLACEHOLDER).expanduser(), "MIMIC_DIR_PLACEHOLDER in cell 1.2"

MIMIC_DIR, MIMIC_DIR_SOURCE = find_mimic_dir()

# Where this notebook writes its own outputs. Nothing is ever written back into MIMIC_DIR.
PROJECT_DIR = Path.cwd() / "readmission_project"
CACHE_DIR = PROJECT_DIR / "cache" # intermediate tables, so slow steps run only once
# (ROW-LEVEL patient data: it stays on your machine)
FIG_DIR = PROJECT_DIR / "figures" # PNG figures for reports and slides
MODEL_DIR = PROJECT_DIR / "models" # fitted pipelines + run_record.json (trained on
# credentialed data, so keep these local too)

for d in (PROJECT_DIR, CACHE_DIR, FIG_DIR, MODEL_DIR):
    d.mkdir(parents=True, exist_ok=True)

# --- Study design knobs -----
READMIT_WINDOW_DAYS = 30 # the prediction horizon. Try 7 or 90 as an exercise.
MIN_AGE = 18 # adults only
REQUIRE_ICU = True # index hospitalisation must contain >= 1 ICU stay

# --- Optional (slow) enrichment -----
# Both stream a multi-gigabyte file in chunks and then cache the small result.
# For a first run both can be set to False; enable them once the pipeline runs end to end.
USE_LABS = True # last lab values before discharge (~10-25 min first run)
USE_PRESCRIPTIONS = True # discharge polypharmacy count (~3-8 min first run)
CHUNKSIZE = 2_000_000 # rows per chunk when streaming. Lower this if you run out of RAM.

# --- Modelling knobs -----
RANDOM_SEED = 42
TEST_SIZE = 0.20 # fraction of *patients* (not rows) held out
N_CV_FOLDS = 5
USE_RACE_AS_FEATURE = False # see Section 11 for why this defaults to False
GRAYSCALE_FIGURES = True # grayscale figures, suitable for printing

np.random.seed(RANDOM_SEED)

# --- Figure styling -----
plt.rcParams.update({

```

```

"figure.dpi": 110,
"savefig.dpi": 200,
"savefig.bbox": "tight",
"figure.facecolor": "white",
"axes.grid": True,
"grid.alpha": 0.25,
"grid.linestyle": ":",
"axes.spines.top": False,
"axes.spines.right": False,
"font.size": 11,
})

# A small palette + linestyle cycle. Using both means every figure still reads
# correctly when printed in black and white or shown on a grayscale slide.
if GRAYSCALE_FIGURES:
    SERIES_STYLE = [
        {"color": "#000000", "linestyle": "-"},
        {"color": "#555555", "linestyle": "--"},
        {"color": "#888888", "linestyle": "-."},
        {"color": "#aaaaaa", "linestyle": ":"},
        {"color": "#333333", "linestyle": (0, (3, 1, 1, 1))},
    ]
    BAR_COLOR = "#444444"
else:
    SERIES_STYLE = [
        {"color": c, "linestyle": ls}
        for c, ls in zip(["#1b6ca8", "#c0392b", "#27743f", "#8e44ad", "#d68910"],
                        ["-", "--", "-.", ":", (0, (3, 1, 1, 1))])
    ]
    BAR_COLOR = "#1b6ca8"

def style(i):
    """Return the plot kwargs for the i-th series (wraps around)."""
    return SERIES_STYLE[i % len(SERIES_STYLE)]

def rel(path):
    """A path relative to the notebook's folder, for printing.

    Printing absolute paths would write your user name and folder layout into the notebook's
    saved output, and from there into anything you share (GitHub, peer review, screenshots).
    """
    try:
        return Path(path).resolve().relative_to(Path.cwd().resolve()).as_posix()
    except ValueError:
        return Path(path).name

def save_fig(fig, name):
    """Save a figure into FIG_DIR under a predictable name and report the path.

    The slide deck refers to these exact filenames, so keep them stable.
    """
    path = FIG_DIR / f"{name}.png"
    fig.savefig(path)
    print(f" saved -> {rel(path)}")
    return path

print(f"MIMIC-IV folder taken from: {MIMIC_DIR_SOURCE}")
print(f"Outputs will be written to: {rel(PROJECT_DIR)}/ (next to this notebook)")

# -----
# 1.3 Locate the data files and fail loudly if the path is wrong
# -----
# MIMIC-IV ships as two module folders of gzipped CSVs:
# hosp/ -> hospital-wide data (admissions, labs, diagnoses, medications, ...)
# icu/ -> ICU-specific data (icustays, chartevents, inputevents, ...)

```

```
# Depending on how you downloaded it, files may be `.csv.gz` or already decompressed `.csv`.
```

```
MODULES = ("hosp", "icu")
```

```
def resolve_table(name):
```

```
    """Return the Path to a MIMIC table, accepting .csv.gz or .csv, in hosp/ or icu/.
```

```
    Parameters
```

```
    -----
```

```
    name : str
```

```
        Bare table name, e.g. "admissions" or "icustays".
```

```
    Raises
```

```
    -----
```

```
    FileNotFoundError
```

```
        With a message that tells the reader exactly what to fix.
```

```
    """
```

```
    for module in MODULES:
```

```
        for ext in (".csv.gz", ".csv"):
```

```
            candidate = MIMIC_DIR / module / f"{name}{ext}"
```

```
            if candidate.exists():
```

```
                return candidate
```

```
    raise FileNotFoundError(
```

```
        f"Could not find table '{name}' under {MIMIC_DIR}.\n"
```

```
        f"Expected something like {MIMIC_DIR / 'hosp' / (name + '.csv.gz')}. \n"
```

```
        f"Check that MIMIC_DIR points at the folder that CONTAINS hosp/ and icu/."
```

```
)
```

```
if not MIMIC_DIR.exists():
```

```
    raise FileNotFoundError(
```

```
        f"MIMIC_DIR does not exist: {MIMIC_DIR} (taken from {MIMIC_DIR_SOURCE})\n"
```

```
        "Set the MIMIC_DIR environment variable, create mimic_path.txt next to this notebook, "
```

```
        "or edit MIMIC_DIR_PLACEHOLDER in cell 1.2 - then re-run from the top."
```

```
)
```

```
# Print only the folder's name, not its full path (see rel() in 1.2 for why).
```

```
print(f"MIMIC-IV root: ../{MIMIC_DIR.name}\n")
```

```
for module in MODULES:
```

```
    mdir = MIMIC_DIR / module
```

```
    if not mdir.exists():
```

```
        print(f" [!] missing module folder: {mdir}")
```

```
        continue
```

```
    files = sorted(p for p in mdir.iterdir() if p.suffix in {".gz", ".csv"})
```

```
    total_gb = sum(p.stat().st_size for p in files) / 1e9
```

```
    print(f" {module}/ {len(files)} files, {total_gb:.1f} GB on disk")
```

```
    for p in files[:6]:
```

```
        print(f" {p.name:<28} {p.stat().st_size / 1e6:8.1f} MB")
```

```
    if len(files) > 6:
```

```
        print(f" ... and {len(files) - 6} more")
```

```
    print()
```

```
# -----
```

```
# 1.4 A thin reader with caching
```

```
# -----
```

```
# Two helpers used everywhere below:
```

```
# read_table() - load a whole (small) table, with explicit dtypes and date parsing
```

```
# cached() - run an expensive function once, then read the parquet/csv next time
```

```
#
```

```
# Specifying `usecols` and `dtype` explicitly matters: on tables with tens of
```

```
# millions of rows it substantially reduces memory use and loading time.
```

```
def read_table(name, usecols=None, dtype=None, parse_dates=None, nrows=None, **kwargs):
```

```
    """Read one MIMIC table into a DataFrame."""
```

```
    path = resolve_table(name)
```

```
    t0 = time.time()
```

```
    df = pd.read_csv(
```

```
        path,
```

```

        usecols=usecols,
        dtype=dtype,
        parse_dates=parse_dates,
        nrows=nrows,
        low_memory=False,
        **kwargs,
    )
    print(f" {name:<18} {len(df):>10,} rows x {df.shape[1]:>2} cols  ({time.time() - t0:5.1f}s)")
    return df

def iter_table(name, usecols=None, dtype=None, parse_dates=None, chunksize=CHUNKSIZE):
    """Yield a huge MIMIC table in chunks so it never has to fit in memory at once."""
    path = resolve_table(name)
    return pd.read_csv(
        path,
        usecols=usecols,
        dtype=dtype,
        parse_dates=parse_dates,
        chunksize=chunksize,
        low_memory=False,
    )

def cached(name, build_fn, force=False):
    """Materialise `build_fn()` to disk once, and reuse it on every later run.

    Prefers parquet (fast, keeps dtypes); falls back to CSV if `pyarrow` is missing.
    Delete the file in CACHE_DIR, or pass force=True, to recompute.
    """
    pq, csv = CACHE_DIR / f"{name}.parquet", CACHE_DIR / f"{name}.csv"
    if not force:
        if pq.exists():
            print(f"[cache] loading {pq.name}")
            return pd.read_parquet(pq)
        if csv.exists():
            print(f"[cache] loading {csv.name}")
            return pd.read_csv(csv, low_memory=False)

    print(f"[cache] building {name} ...")
    t0 = time.time()
    df = build_fn()
    try:
        df.to_parquet(pq, index=False)
        print(f"[cache] wrote {pq.name} in {time.time() - t0:.1f}s")
    except Exception as exc:  # pyarrow/fastparquet not installed
        df.to_csv(csv, index=False)
        print(f"[cache] parquet unavailable ({type(exc).__name__}); wrote {csv.name}")
    return df

print("Helpers ready.")

```

2. Study design

The study design is specified before any data processing. Each element below is stated explicitly because every later decision depends on it.

Question	Answer for this project
Population	Adults (≥ 18 at admission) with a hospitalisation that included at least one ICU stay, who were discharged alive.
Index event	One row per <i>hospital admission</i> (<code>hadm_id</code>), not per ICU stay and not per patient.

Question	Answer for this project
Prediction time	<code>dischtime</code> — the moment of discharge from the index hospitalisation.
Outcome	A later hospital admission for the same patient with an <i>unplanned</i> admission type, starting within 30 days of <code>dischtime</code> .
Horizon	30 days (<code>READMIT_WINDOW_DAYS</code>).
Excluded	In-hospital deaths, discharges to hospice, non-positive length of stay, and patients who died within 30 days without being readmitted (see below).
Intended use	Rank patients at discharge so a transitional-care team can prioritise follow-up calls and early clinic visits. It is a <i>triage aid</i> , never a discharge-blocking decision.
Consequences of error	A false negative is a patient who is readmitted without having received follow-up; a false positive uses limited nursing time for a patient who did not need it. The two errors have different costs.

Key design decisions

1. "Unplanned" readmission, not "any" readmission. A patient who returns a few days after discharge for a scheduled chemotherapy cycle or a staged surgery has not experienced an adverse outcome. Counting such admissions would inflate the outcome rate and train the model to detect *planned care*. A later admission therefore counts as an outcome only if its `admission_type` is one of the emergency, urgent or observation categories.

2. Death is a competing risk. A patient who dies at home on day 9 cannot be readmitted. Labelling such patients "no readmission" would associate the most severe illness with the lowest risk. The approach used here is to **exclude** patients who died within the window without a readmission. As a consequence, the target population becomes "patients who survive 30 days", and the model does not estimate readmission risk for all discharged patients. The methodologically stronger alternative is a competing-risks survival model (Fine-Gray), listed as an extension in Section 14.

3. MIMIC-IV covers a single hospital. If a patient is readmitted to a different hospital, MIMIC-IV has no record of it, and the admission is labelled a non-event. This is *one-directional label noise*: the measured readmission rate is a lower bound. Published 30-day readmission rates after an ICU stay are approximately 10–20%. This limitation applies to every result in this notebook.

3. Load the core tables

Four small tables define the cohort. All four fit in memory.

```
# -----
# 3.1 admissions - one row per hospitalisation
# -----
# Key columns:
#   subject_id      patient identifier (stable across admissions)
#   hadm_id         hospitalisation identifier (our unit of analysis)
#   admittime/dischtime start and end of the hospitalisation
#   deathtime       time of death if the patient died IN hospital
#   admission_type   ELECTIVE / EW EMER. / URGENT / OBSERVATION ADMIT / ...
#   admission_location where the patient came from (ED, physician referral, transfer, ...)
```

```

# discharge_location where they went (HOME, SNF, REHAB, HOSPICE, DIED, ...)
# edregtime/edouttime emergency department arrival and departure, when applicable
# hospital_expire_flag 1 if the patient died during this hospitalisation
print("Loading core tables:")

admissions = read_table(
    "admissions",
    parse_dates=["admittime", "disctime", "deathtime", "edregtime", "edouttime"],
)

# -----
# 3.2 patients - one row per patient
# -----
# MIMIC-IV protects dates by shifting each patient's timeline into the future by a random
# offset, so `admittime` may read 2153. Intervals WITHIN a patient are preserved exactly,
# which is all we need. To recover age:
#
#     age_at_admission = anchor_age + (year(admittime) - anchor_year)
#
# `anchor_year_group` gives the real-world 3-year window that contains the patient's
# anchor_year ("2008 - 2010", ..., "2020 - 2022"). It is coarse and patient-level, but it is
# our only handle on real calendar time, and we use it for the temporal validation in Section 12.
# Patients older than 89 are all recorded as anchor_age 91 for de-identification.
patients = read_table("patients", parse_dates=["dod"])

# -----
# 3.3 icustays - one row per ICU stay (a hospitalisation can have several)
# -----
icustays = read_table("icustays", parse_dates=["intime", "outtime"])

# -----
# 3.4 diagnoses_icd - billing diagnoses, many rows per hospitalisation
# -----
# `seq_num` is the priority order; seq_num == 1 is the principal diagnosis.
# `icd_version` is 9 or 10 - MIMIC-IV spans the U.S. transition, so you MUST handle both.
diagnoses = read_table(
    "diagnoses_icd",
    dtype={"icd_code": "string", "icd_version": "int8", "seq_num": "float32"},
)

# What do these tables look like? We show each table's SCHEMA - column names, types, how
# complete each column is - instead of `.head()`. The reason is the Data Use Agreement:
# notebooks are often shared (for example, for peer review), and a `.head()`
# output is a handful of real patient records saved inside the file. A schema tells a reader
# everything they need to follow along without any row-level data leaving your machine.
def schema(df):
    """One row per column: dtype, % non-missing, number of distinct values. Aggregate only."""
    return pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "non_missing_%": (df.notna().mean() * 100).round(1),
        "n_distinct": df.nunique(dropna=True),
    })

print("\nadmissions - schema:")
display(schema(admissions))
print("patients - schema:")
display(schema(patients))

```

```

# -----
# 3.5 Look before you leap: what values actually appear?
# -----
# Category strings should be verified against the local copy of the data before use.
# Value spellings differ between MIMIC-IV versions (e.g. language was "ENGLISH" in v2.2
# and "English" in v3.x). This cell shows the values actually present.
for col in ["admission_type", "admission_location", "discharge_location", "insurance"]:
    vc = admissions[col].value_counts(dropna=False)

```

```
print(f"\n--- {col} ({vc.shape[0]} distinct) ---")
print(vc.head(14).to_string())
```

```
# -----
# 3.6 Define what counts as an UNPLANNED admission
# -----
# We build this set from what is actually present, so a typo cannot silently produce an
# all-zero label, an error that is otherwise easy to miss.
observed_types = set(admissions["admission_type"].dropna().unique())

UNPLANNED_TYPES = {
    "EW EMER.",          # emergency ward emergency
    "EU EMER.",          # some releases spell it this way
    "DIRECT EMER.",      # direct emergency admission
    "URGENT",
    "OBSERVATION ADMIT",
    "EU OBSERVATION",
    "DIRECT OBSERVATION",
    "AMBULATORY OBSERVATION",
}
UNPLANNED_TYPES &= observed_types # keep only those that exist in this release

PLANNED_TYPES = observed_types - UNPLANNED_TYPES

print("Treated as UNPLANNED (counts as an outcome):")
for t in sorted(UNPLANNED_TYPES):
    print(f"    {t}")
print("\nTreated as PLANNED (does NOT count as an outcome):")
for t in sorted(PLANNED_TYPES):
    print(f"    {t}")

assert UNPLANNED_TYPES, (
    "No admission types matched the unplanned list. Inspect the value counts printed above "
    "and update UNPLANNED_TYPES to match the spellings in your copy of MIMIC-IV."
)
```

4. Build the cohort

We build the cohort as a sequence of filters, recording the row and patient count after each one. That table provides the CONSORT-style flow diagram that reviewers expect. Recording it during cohort construction is more reliable than reconstructing it afterwards.

```
# -----
# 4.1 A tiny helper that records the cohort flow as we filter
# -----
flow = []

def log_step(df, label):
    """Append (step, n_admissions, n_patients) to the flow log and print it."""
    n_adm = len(df)
    n_pat = df["subject_id"].nunique()
    prev = flow[-1] if flow else None
    removed = "" if prev is None else f" (-{prev['n_admissions'] - n_adm:,})"
    flow.append({"step": label, "n_admissions": n_adm, "n_patients": n_pat})
    print(f"{label:<52} {n_adm:>9,} adm {n_pat:>8,} pts{removed}")
    return df

# Start from every hospitalisation in MIMIC-IV. (The flow log is filled in in 4.3,
# after we have attached patient attributes, so that re-running a cell cannot
# corrupt it.)
```

```
cohort = admissions.copy()
print(f"Starting point: {len(cohort):,} admissions")
```

```
# -----
# 4.2 Attach patient-level attributes and compute age at admission
# -----
cohort = cohort.merge(
    patients[["subject_id", "gender", "anchor_age", "anchor_year", "anchor_year_group", "dod"]],
    on="subject_id",
    how="left",
    validate="many_to_one", # every admission maps to exactly one patient row
)

# Recover age (see the note in 3.2). `.dt.year` on the shifted date is fine because
# anchor_year is shifted by the same per-patient offset.
cohort["age"] = cohort["anchor_age"] + (cohort["admittime"].dt.year - cohort["anchor_year"])

# Length of stay, in days, as a float. This is a *feature*, known at discharge.
cohort["los_days"] = (cohort["disctime"] - cohort["admittime"]).dt.total_seconds() / 86400.0

# Emergency department dwell time, in hours. NaN when the patient did not come via the ED,
# which is itself informative - we will keep a "came via ED" flag rather than silently
# imputing a zero.
cohort["ed_los_hours"] = (cohort["edouttime"] - cohort["edregtime"]).dt.total_seconds() / 3600.0
cohort["came_via_ed"] = cohort["edregtime"].notna().astype("int8")

print(cohort[["age", "los_days", "ed_los_hours"]].describe().round(2).to_string())
```

```
# -----
# 4.3 Apply the inclusion / exclusion criteria, one at a time
# -----
flow.clear() # makes this cell safe to re-run
print(f"{'STEP':<52} {'ADMISSIONS':>9} {'PATIENTS':>8}")
print("-" * 86)
log_step(cohort, "0. All hospital admissions")

# --- (a) adults only -----
# MIMIC-IV should contain no paediatric patients; this filter makes the criterion explicit.
cohort = cohort[cohort["age"] >= MIN_AGE]
log_step(cohort, f"1. Age >= {MIN_AGE} at admission")

# --- (b) must include an ICU stay -----
# We aggregate icustays to one row per hadm_id first: total ICU time, number of stays,
# and the first/last care unit. These double as features later.
icu_agg = (
    icustays.groupby("hadm_id")
    .agg(
        n_icu_stays=("stay_id", "nunique"),
        icu_los_days=("los", "sum"),
        icu_los_max=("los", "max"),
        first_icu_intime=("intime", "min"),
        last_icu_outtime=("outtime", "max"),
    )
    .reset_index()
)
first_unit = (
    icustays.sort_values(["hadm_id", "intime"])
    .groupby("hadm_id")
    .agg(first_careunit=("first_careunit", "first"), last_careunit=("last_careunit", "last"))
    .reset_index()
)
icu_agg = icu_agg.merge(first_unit, on="hadm_id", how="left")

cohort = cohort.merge(icu_agg, on="hadm_id", how="left" if not REQUIRE_ICU else "inner")
log_step(cohort, "2. Hospitalisation included an ICU stay")

# --- (c) discharged alive -----
# Two independent signals are checked and both must indicate survival, because a
```

```
# misrecorded death would otherwise become a false negative.
cohort = cohort[(cohort["hospital_expire_flag"] == 0) & (cohort["deathtime"].isna())]
cohort = cohort[cohort["discharge_location"].fillna("") != "DIED"]
log_step(cohort, "3. Discharged alive")

# --- (d) not discharged to hospice -----
# A hospice discharge is a transition to comfort care. Readmission is not the relevant
# outcome for these patients, and including them would teach the model that terminal
# illness predicts a low readmission risk.
cohort = cohort[~cohort["discharge_location"].fillna("").str.contains("HOSPICE", na=False)]
log_step(cohort, "4. Not discharged to hospice")

# --- (e) plausible length of stay -----
cohort = cohort[(cohort["los_days"] > 0) & (cohort["los_days"] < 365)]
log_step(cohort, "5. Length of stay in (0, 365) days")

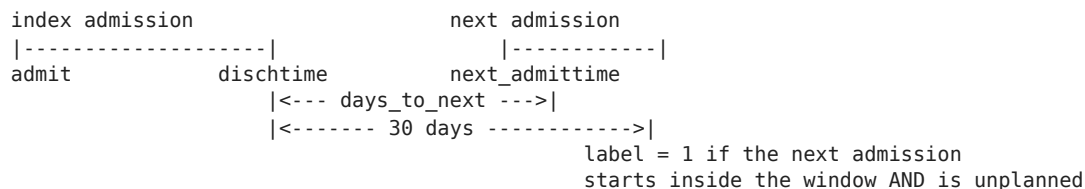
cohort = cohort.reset_index(drop=True)
```

4.4 Create the label

Label construction is a common source of error, so each step is shown explicitly.

For each patient we sort their admissions in time and look at the **next** one. Three facts about that next admission decide the label:

1. How long after discharge did it start? (`days_to_next`)
2. Was it unplanned?
3. Did the patient die before the window closed?



Note we use the **next** admission only. If the next admission is an elective one on day 3 and an emergency one follows on day 10, our rule records a 0. That is a deliberate, documented simplification; Exercise 4 in Section 15 relaxes it.

```
# -----
# 4.4 Label construction
# -----
# IMPORTANT: look up the *next* admission in the FULL admissions table, not in the filtered
# cohort. A readmission that was elective, paediatric, or non-ICU is still a readmission -
# it just may not qualify as an outcome. Filtering first would hide it entirely.
nxt = admissions[["subject_id", "hadm_id", "admittime", "admission_type"]].copy()
nxt = nxt.sort_values(["subject_id", "admittime"])

# Within each patient, shift the admission columns up by one row to get "the next admission".
nxt["next_hadm_id"] = nxt.groupby("subject_id")["hadm_id"].shift(-1)
nxt["next_admittime"] = nxt.groupby("subject_id")["admittime"].shift(-1)
nxt["next_admission_type"] = nxt.groupby("subject_id")["admission_type"].shift(-1)

cohort = cohort.merge(
    nxt[["next_hadm_id", "next_admittime", "next_admission_type"]],
    on="subject_id",
    how="left",
    validate="one_to_one",
)

# Days from discharge to the start of the next admission.
```

```

cohort["days_to_next"] = (
    (cohort["next_admittime"] - cohort["disctime"]).dt.total_seconds() / 86400.0
)

# Days from discharge to death (NaT -> NaN -> "did not die in our records").
cohort["days_to_death"] = (
    (cohort["dod"] - cohort["disctime"].dt.normalize()).dt.total_seconds() / 86400.0
)

# --- Sanity check: a few negative values are expected -----
# Overlapping or nested admissions exist in real EHR data (transfers coded as separate
# hadm_ids, administrative corrections). We treat a "next admission" that starts before
# discharge as not a readmission.
n_negative = (cohort["days_to_next"] < 0).sum()
print(f"Next admission starts before discharge (overlapping records): {n_negative:,}")

# --- The label -----
in_window = cohort["days_to_next"].between(0, READMIT_WINDOW_DAYS, inclusive="both")
is_unplanned = cohort["next_admission_type"].isin(UNPLANNED_TYPES)
cohort["readmit_30d"] = (in_window & is_unplanned).astype("int8")

# --- Competing risk: died within the window, never readmitted -----
died_in_window = (
    cohort["days_to_death"].notna()
    & (cohort["days_to_death"] >= 0)
    & (cohort["days_to_death"] <= READMIT_WINDOW_DAYS)
)
cohort["died_within_window"] = (died_in_window & (cohort["readmit_30d"] == 0)).astype("int8")

print(f"\nBefore competing-risk exclusion: {len(cohort):,} admissions, "
      f"{cohort['readmit_30d'].mean():.1%} positive")
print(f"Died within {READMIT_WINDOW_DAYS} days without readmission: "
      f"{cohort['died_within_window'].sum():,}")

cohort = cohort[cohort["died_within_window"] == 0].reset_index(drop=True)
log_step(cohort, f"6. Survived {READMIT_WINDOW_DAYS}d or was readmitted")

print(f"\nFINAL COHORT: {len(cohort):,} admissions from "
      f"{cohort['subject_id'].nunique():,} patients")
print(f"Outcome rate: {cohort['readmit_30d'].mean():.2%} "
      f"({int(cohort['readmit_30d'].sum()):,} readmissions)")

```

```

# -----
# 4.5 The cohort flow table - put this on a slide
# -----
flow_df = pd.DataFrame(flow).drop_duplicates(subset="step", keep="last").reset_index(drop=True)
flow_df["pct_of_start"] = (flow_df["n_admissions"] / flow_df["n_admissions"].iloc[0] * 100).round(1)
display(flow_df)
flow_df.to_csv(PROJECT_DIR / "cohort_flow.csv", index=False)

# A horizontal bar chart of the same information.
fig, ax = plt.subplots(figsize=(8, 3.6))
ax.barh(flow_df["step"][::-1], flow_df["n_admissions"][::-1], color=BAR_COLOR, height=0.6)
for y, v in enumerate(flow_df["n_admissions"][::-1]):
    ax.text(v, y, f" {v:,}", va="center", fontsize=9)
ax.set_xlabel("Hospital admissions remaining")
ax.set_title("Cohort construction flow")
ax.grid(axis="y", visible=False)
ax.margins(x=0.18)
save_fig(fig, "fig01_cohort_flow")
plt.show()

```

```

# -----
# 4.6 Verify the label before trusting it
# -----
# Three inexpensive checks that detect the most common label-construction errors.

# (1) The distribution of days_to_next among positives must lie inside the window.

```

```

pos = cohort[cohort["readmit_30d"] == 1]
assert pos["days_to_next"].between(0, READMIT_WINDOW_DAYS).all(), "positives outside the window!"
print(f"(1) days_to_next for positives: min={pos['days_to_next'].min():.2f}, "
      f"max={pos['days_to_next'].max():.2f} OK")

# (2) Every positive's next admission type must be in our unplanned set.
assert pos["next_admission_type"].isin(UNPLANNED_TYPES).all()
print("(2) all positive next-admission types are unplanned OK")

# (3) The outcome rate should be clinically plausible (roughly 5-30% for this population).
rate = cohort["readmit_30d"].mean()
print(f"(3) outcome rate = {rate:.2%} "
      f"{'-- plausible' if 0.03 < rate < 0.35 else '-- SUSPICIOUS, investigate'}")

# Time-to-readmission histogram: readmissions cluster in the first week, which is the
# well-documented pattern and consistent with a correctly constructed label.
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.hist(pos["days_to_next"], bins=READMIT_WINDOW_DAYS, color=BAR_COLOR, edgecolor="white")
ax.set_xlabel("Days from discharge to unplanned readmission")
ax.set_ylabel("Number of readmissions")
ax.set_title(f"Time to unplanned readmission (n={len(pos):,})")
save_fig(fig, "fig02_time_to_readmission")
plt.show()

```

5. Feature engineering

Principle

Every feature must be computable at `disctime` using only information recorded on or before `disctime`.

Violating this principle is called **target leakage**. Leakage produces performance estimates that cannot be reproduced in deployment, and it is often detected only through implausibly high metrics (for example, an AUROC near 0.95 for this task).

Three examples of leakage that could arise in this project:

Candidate feature	Why it leaks
"Number of admissions this patient has"	Counts admissions in the <i>future</i> too. A patient with many admissions has them partly <i>because</i> they were readmitted. Must be counted strictly before the index admission.
"Time until the patient's death"	Uses <code>dod</code> , recorded after discharge.
"Average length of stay for this patient"	Same problem: averages over future encounters.

History features are therefore computed with an explicit backward-looking window, and a test confirms that they use only earlier admissions.

```

# -----
# 5.1 Prior-utilisation features (the leakage-sensitive ones)
# -----
# Strategy: work on the FULL admissions table (so nothing is hidden), sort within patient,
# and use a backward-looking rolling time window.
#
# pandas' `.rolling("365D")` on a datetime index counts every row whose index falls in
# (t - 365 days, t]. That INCLUDES the current row, so we subtract 1 to get "prior".

```

```

hist = admissions[["subject_id", "hadm_id", "admittime", "dischtime", "admission_type"]].copy()
hist = hist.sort_values(["subject_id", "admittime"]).reset_index(drop=True)
hist["is_unplanned"] = hist["admission_type"].isin(UNPLANNED_TYPES).astype("int8")
hist["ones"] = 1

indexed = hist.set_index("admittime")

# (a) admissions in the preceding 365 days
rolled_all = indexed.groupby("subject_id")["ones"].rolling("365D").sum()
# (b) *unplanned* admissions in the preceding 365 days
rolled_unp = indexed.groupby("subject_id")["is_unplanned"].rolling("365D").sum()

# groupby preserves within-group order and iterates groups in sorted subject_id order,
# which is exactly the order of `hist`. The assertion below verifies this.
assert len(rolled_all) == len(hist), "rolling result misaligned with the admissions table"

hist["n_prior_admits_365d"] = rolled_all.to_numpy() - 1
hist["n_prior_unplanned_365d"] = rolled_unp.to_numpy() - hist["is_unplanned"].to_numpy()

# (c) lifetime count of strictly earlier admissions (cumcount is 0 for the first)
hist["n_prior_admits_total"] = hist.groupby("subject_id").cumcount()

# (d) days since the patient's previous discharge (NaN for a first-ever admission)
prev_disch = hist.groupby("subject_id")["dischtime"].shift(1)
hist["days_since_prev_discharge"] = (
    (hist["admittime"] - prev_disch).dt.total_seconds() / 86400.0
)

# (e) a simple flag: has this patient ever been here before?
hist["is_first_admission"] = (hist["n_prior_admits_total"] == 0).astype("int8")

prior_cols = [
    "hadm_id", "n_prior_admits_365d", "n_prior_unplanned_365d",
    "n_prior_admits_total", "days_since_prev_discharge", "is_first_admission",
]
cohort = cohort.merge(hist[prior_cols], on="hadm_id", how="left", validate="one_to_one")

# --- Leakage test -----
# For a random sample of admissions, recount prior admissions the slow, obviously-correct
# way and confirm that the fast version agrees.
rng = np.random.default_rng(RANDOM_SEED)
sample_idx = rng.choice(len(cohort), size=min(200, len(cohort)), replace=False)
sample_subjects = set(cohort.iloc[sample_idx]["subject_id"])
sub_adm = admissions[admissions["subject_id"].isin(sample_subjects)]
adm_by_subject = {sid: g["admittime"].to_numpy()
                  for sid, g in sub_adm.groupby("subject_id")}
mismatches = 0
for i in sample_idx:
    row = cohort.iloc[i]
    times = adm_by_subject[row["subject_id"]]
    t = np.datetime64(row["admittime"])
    slow = int(((times < t) & (times > t - np.timedelta64(365, "D"))).sum())
    if slow != int(row["n_prior_admits_365d"]):
        mismatches += 1
print(f"Leakage/correctness test on {len(sample_idx)} sampled admissions: "
      f"{mismatches} mismatches {'-- PASS' if mismatches == 0 else '-- INVESTIGATE'}")

display(cohort[["n_prior_admits_365d", "n_prior_unplanned_365d",
                "n_prior_admits_total", "days_since_prev_discharge"]].describe().round(2))

```

5.2 Comorbidity burden: the Charlson Comorbidity Index

Raw ICD codes are impractical as model inputs: there are roughly 70,000 of them, most of which appear only a few times. The standard approach in health-services research is to collapse them into validated **comorbidity groups**. We implement the Charlson Comorbidity Index using the coding algorithm published by Quan et al. (*Medical Care*, 2005), which gives

code lists for both ICD-9-CM and ICD-10. This is necessary here because MIMIC-IV spans the U.S. transition between them and a single hospitalisation can contain either.

The index is a weighted sum of 17 condition groups (weights 1, 2, 3, 6), originally derived to predict 1-year mortality. Three hierarchy rules avoid double counting:

- diabetes **with** complications supersedes diabetes without,
- severe liver disease supersedes mild,
- metastatic solid tumour supersedes non-metastatic malignancy.

Scope. The implementation below is *simplified* and written for readability rather than completeness: we match on code prefixes, which occasionally over-captures a neighbouring code. For publication-grade work, use the validated SQL in the MIMIC code repository (`concepts/comorbidity/charlson.sql`) and compare against this. That comparison is a useful exercise.

```
# -----
# 5.2 Charlson Comorbidity Index from ICD-9 and ICD-10 codes
# -----
def icd9_range(lo, hi):
    """Expand an inclusive ICD-9 3-digit range, e.g. 140-172 -> ['140','141',...,'172']. """
    return [f"{i:03d}" for i in range(lo, hi + 1)]

def icd10_range(letter, lo, hi):
    """Expand an inclusive ICD-10 range, e.g. ('C', 0, 26) -> ['C00','C01',...,'C26']. """
    return [f"{letter}{i:02d}" for i in range(lo, hi + 1)]

# Each entry: weight, plus the ICD-9-CM and ICD-10 code prefixes that define the group.
# Prefixes are written at their natural length (3, 4 or 5 characters); the matcher below
# compares each length separately, so mixing lengths is fine.
CHARLSON = {
    "myocardial_infarction": dict(
        weight=1,
        icd9=["410", "412"],
        icd10=["I21", "I22", "I252"]),
    "congestive_heart_failure": dict(
        weight=1,
        icd9=["39891", "40201", "40211", "40291", "40401", "40403", "40411", "40413",
              "40491", "40493", "4254", "4255", "4256", "4257", "4258", "4259", "428"],
        icd10=["I099", "I110", "I130", "I132", "I255", "I420", "I425", "I426", "I427",
              "I428", "I429", "I43", "I50", "P290"]),
    "peripheral_vascular": dict(
        weight=1,
        icd9=["0930", "4373", "440", "441", "4431", "4432", "4433", "4434", "4435",
              "4436", "4437", "4438", "4439", "4471", "5571", "5579", "V434"],
        icd10=["I70", "I71", "I731", "I738", "I739", "I771", "I790", "I792",
              "K551", "K558", "K559", "Z958", "Z959"]),
    "cerebrovascular": dict(
        weight=1,
        icd9=["36234"] + icd9_range(430, 438),
        icd10=["G45", "G46", "H340"] + icd10_range("I", 60, 69)),
    "dementia": dict(
        weight=1,
        icd9=["290", "2941", "3312"],
        icd10=["F00", "F01", "F02", "F03", "F051", "G30", "G311"]),
    "chronic_pulmonary": dict(
        weight=1,
        icd9=["4168", "4169"] + icd9_range(490, 505) + ["5064", "5081", "5088"],
        icd10=["I278", "I279"] + icd10_range("J", 40, 47) + icd10_range("J", 60, 67)
              + ["J684", "J701", "J703"]),
    "rheumatic": dict(
```

```

    weight=1,
    icd9=["4465", "7100", "7101", "7102", "7103", "7104",
          "7140", "7141", "7142", "7148", "725"],
    icd10=["M05", "M06", "M315", "M32", "M33", "M34", "M351", "M353", "M360"]),
"peptic_ulcer": dict(
    weight=1,
    icd9=["531", "532", "533", "534"],
    icd10=["K25", "K26", "K27", "K28"]),
"mild_liver": dict(
    weight=1,
    icd9=["07022", "07023", "07032", "07033", "07044", "07054", "0706", "0709",
          "570", "571", "5733", "5734", "5738", "5739", "V427"],
    icd10=["B18", "K700", "K701", "K702", "K703", "K709", "K713", "K714", "K715",
          "K717", "K73", "K74", "K760", "K762", "K763", "K764", "K768", "K769",
          "Z944"]),
"diabetes_uncomplicated": dict(
    weight=1,
    icd9=["2500", "2501", "2502", "2503", "2508", "2509"],
    icd10=["E100", "E101", "E106", "E108", "E109", "E110", "E111", "E116", "E118",
          "E119", "E120", "E121", "E126", "E128", "E129", "E130", "E131", "E136",
          "E138", "E139", "E140", "E141", "E146", "E148", "E149"]),
"diabetes_complicated": dict(
    weight=2,
    icd9=["2504", "2505", "2506", "2507"],
    icd10=["E102", "E103", "E104", "E105", "E107", "E112", "E113", "E114", "E115",
          "E117", "E122", "E123", "E124", "E125", "E127", "E132", "E133", "E134",
          "E135", "E137", "E142", "E143", "E144", "E145", "E147"]),
"paralysis": dict(
    weight=2,
    icd9=["3341", "342", "343", "3440", "3441", "3442", "3443", "3444", "3445",
          "3446", "3449"],
    icd10=["G041", "G114", "G801", "G802", "G81", "G82", "G830", "G831", "G832",
          "G833", "G834", "G839"]),
"renal_disease": dict(
    weight=2,
    icd9=["40301", "40311", "40391", "40402", "40403", "40412", "40413", "40492",
          "40493", "582", "5830", "5831", "5832", "5834", "5836", "5837", "585",
          "586", "5880", "V420", "V451", "V56"],
    icd10=["I120", "I131", "N032", "N033", "N034", "N035", "N036", "N037", "N052",
          "N053", "N054", "N055", "N056", "N057", "N18", "N19", "N250", "Z490",
          "Z491", "Z492", "Z940", "Z992"]),
"malignancy": dict(
    weight=2,
    icd9=icd9_range(140, 172) + icd9_range(174, 195) + icd9_range(200, 208) + ["2386"],
    icd10=(icd10_range("C", 0, 26) + icd10_range("C", 30, 34) + icd10_range("C", 37, 41)
          + ["C43"] + icd10_range("C", 45, 58) + icd10_range("C", 60, 76)
          + icd10_range("C", 81, 85) + ["C88"] + icd10_range("C", 90, 97))),
"severe_liver": dict(
    weight=3,
    icd9=["4560", "4561", "4562", "5722", "5723", "5724", "5728"],
    icd10=["I850", "I859", "I864", "I982", "K704", "K711", "K721", "K729",
          "K765", "K766", "K767"]),
"metastatic_cancer": dict(
    weight=6,
    icd9=["196", "197", "198", "199"],
    icd10=["C77", "C78", "C79", "C80"]),
"aids_hiv": dict(
    weight=6,
    icd9=["042", "043", "044"],
    icd10=["B20", "B21", "B22", "B24"]),
}

def _by_length(codes):
    """Group a list of prefixes by their character length -> {len: set(prefixes)}."""
    out = {}
    for c in codes:
        out.setdefault(len(c), set()).add(c)
    return out

```

```

def _match(series, prefixes_by_len):
    """Boolean mask: does each code start with any prefix in the given set?

    We compare fixed-length slices against a Python set, which is far faster than
    a regex or a chain of str.startswith calls on millions of rows.
    """
    mask = np.zeros(len(series), dtype=bool)
    for length, prefix_set in prefixes_by_len.items():
        mask |= series.str[:length].isin(prefix_set).to_numpy(dtype=bool)
    return pd.Series(mask, index=series.index)

def compute_charlson(dx, hadm_ids):
    """Return one row per hadm_id with 17 comorbidity flags and the Charlson score.

    Parameters
    -----
    dx : DataFrame with columns [hadm_id, icd_code, icd_version]
    hadm_ids : iterable of hadm_id values to restrict to (the cohort)
    """
    d = dx[dx["hadm_id"].isin(set(hadm_ids))].copy()
    # Normalise: upper case, strip padding whitespace. MIMIC stores ICD-9 without the
    # decimal point already (414.01 -> "41401"), which is what our prefixes assume.
    d["code"] = d["icd_code"].fillna("").astype(str).str.upper().str.strip()
    is9, is10 = d["icd_version"] == 9, d["icd_version"] == 10

    flags = pd.DataFrame({"hadm_id": d["hadm_id"]})
    for name, spec in CHARLSON.items():
        m9 = _match(d["code"], _by_length(spec["icd9"])) & is9
        m10 = _match(d["code"], _by_length(spec["icd10"])) & is10
        flags[name] = (m9 | m10).astype("int8")

    # Collapse the many diagnosis rows per hospitalisation into one row of flags.
    out = flags.groupby("hadm_id").max().reset_index()

    # Hierarchy rules: do not count the milder form when the severe form is present.
    out.loc[out["diabetes_complicated"] == 1, "diabetes_uncomplicated"] = 0
    out.loc[out["severe_liver"] == 1, "mild_liver"] = 0
    out.loc[out["metastatic_cancer"] == 1, "malignancy"] = 0

    out["charlson_score"] = sum(out[n] * s["weight"] for n, s in CHARLSON.items())
    return out

charlson = cached(
    "charlson",
    lambda: compute_charlson(diagnoses, cohort["hadm_id"]),
)

# Total number of coded diagnoses is itself a crude proxy for complexity / coding intensity.
n_dx = (
    diagnoses[diagnoses["hadm_id"].isin(set(cohort["hadm_id"]))]
    .groupby("hadm_id").size().rename("n_diagnoses").reset_index()
)

cohort = cohort.merge(charlson, on="hadm_id", how="left").merge(n_dx, on="hadm_id", how="left")

# Admissions with no coded diagnoses get 0, not NaN - absence of a code means absence of
# a recorded condition, which is the appropriate interpretation for billing data.
charlson_cols = list(CHARLSON.keys()) + ["charlson_score"]
cohort[charlson_cols + ["n_diagnoses"]] = cohort[charlson_cols + ["n_diagnoses"]].fillna(0)

print(f"Charlson score: mean={cohort['charlson_score'].mean():.2f}, "
      f"median={cohort['charlson_score'].median():.0f}, "
      f"max={cohort['charlson_score'].max():.0f}")
print("\nPrevalence of each comorbidity group, and readmission rate within it:")
summary = pd.DataFrame({

```

```

    "prevalence": cohort[list(CHARLSON.keys())].mean(),
    "readmit_rate_if_present": [
        cohort.loc[cohort[c] == 1, "readmit_30d"].mean() for c in CHARLSON
    ],
}).sort_values("prevalence", ascending=False)
display((summary * 100).round(1).rename(columns=lambda c: c + " (%)"))

```

5.3 Optional: last laboratory values before discharge

`labevents` is the largest table in the `hosp` module (over 100 million rows). Loading it at once exceeds the memory of a typical laptop. The standard approach is to **stream, filter, and discard**: read a chunk, keep only the rows that are needed (well under 1%), and continue with the next chunk. The small result is cached so that this step runs only once.

We take the **last recorded value before discharge** for a panel of twelve common analytes. The last creatinine before discharge, for instance, is informative: incomplete recovery of kidney function at discharge is a recognised readmission risk factor.

Set `USE_LABS = False` in the configuration cell to skip this entirely ; the rest of the notebook runs without it.

```

# -----
# 5.3 Streaming extraction of last-before-discharge lab values
# -----
# itemids come from hosp/d_labitems.csv.gz. These are the standard MIMIC-IV blood
# chemistry / haematology itemids; verify them against your own copy (we print the
# labels below so you can check).
LAB_ITEMS = {
    50912: "creatinine",
    51006: "bun",
    50983: "sodium",
    50971: "potassium",
    50882: "bicarbonate",
    50868: "anion_gap",
    51222: "hemoglobin",
    51301: "wbc",
    51265: "platelets",
    50931: "glucose",
    50862: "albumin",
    50885: "bilirubin_total",
}

# Confirm that the itemids correspond to the intended analytes before using them.
d_lab = read_table("d_labitems")
display(d_lab[d_lab["itemid"].isin(LAB_ITEMS)][["itemid", "label", "fluid", "category"]])

def extract_last_labs():
    """Stream labevents and keep the last value before discharge for each (hadm_id, itemid)."""
    wanted_hadm = set(cohort["hadm_id"])
    disch = cohort.set_index("hadm_id")["disctime"]
    keep = []
    rows_seen = 0
    t0 = time.time()

    for i, chunk in enumerate(iter_table(
        "labevents",
        usecols=["hadm_id", "itemid", "charttime", "valuenum"],
        dtype={"hadm_id": "float64", "itemid": "int32", "valuenum": "float32"},
        parse_dates=["charttime"],
    )):
        rows_seen += len(chunk)
        # Filter early: this keeps memory use low.
        chunk = chunk[chunk["itemid"].isin(LAB_ITEMS)]

```

```

    chunk = chunk[chunk["hadm_id"].isin(wanted_hadm)]
    chunk = chunk.dropna(subset=["valuenum", "charttime"])
    if len(chunk):
        keep.append(chunk)
    if (i + 1) % 5 == 0:
        print(f"    chunk {i + 1}: {rows_seen:,} rows scanned, "
              f"{sum(len(k) for k in keep):,} kept, {time.time() - t0:.0f}s")

labs = pd.concat(keep, ignore_index=True)
labs["hadm_id"] = labs["hadm_id"].astype("int64")

# Drop anything charted after discharge (rare, but it would be leakage).
labs = labs.merge(disch.rename("disctime"), left_on="hadm_id", right_index=True, how="left")
labs = labs[labs["charttime"] <= labs["disctime"]]

# Last value per hospitalisation per analyte.
labs = labs.sort_values("charttime")
last = labs.groupby(["hadm_id", "itemid"])["valuenum"].last().unstack()
last.columns = [f"last_{LAB_ITEMS[c]}" for c in last.columns]
return last.reset_index()

if USE_LABS:
    last_labs = cached("last_labs", extract_last_labs)
    cohort = cohort.merge(last_labs, on="hadm_id", how="left")
    # Take the column list from the lab table itself. Do NOT scan cohort.columns for a
    # "last_" prefix: icustays already contributed `last_careunit` and `last_icu_outtime`,
    # and sweeping those into the numeric feature list produces a duplicated-column error
    # several cells later that is difficult to trace back to this cell.
    lab_cols = [c for c in last_labs.columns if c != "hadm_id"]
    print(f"\nAdded {len(lab_cols)} lab features.")
    display(cohort[lab_cols].describe().T[["count", "mean", "50%", "std"]].round(2))
    print("\nMissingness is informative here: whether a lab was drawn carries information "
          "about the patient. We impute it later, but we also keep a 'was it measured' flag.")
else:
    lab_cols = []
    print("USE_LABS is False - skipping laboratory features.")

# -----
# 5.4 Optional: polypharmacy (number of distinct drugs ordered during the stay)
# -----
# Patients leaving on many medications are at higher risk of drug interactions, confusion
# about the regimen, and non-adherence, all established readmission risk factors.
def extract_drug_counts():
    wanted = set(cohort["hadm_id"])
    parts = []
    for chunk in iter_table(
        "prescriptions",
        usecols=["hadm_id", "drug"],
        dtype={"hadm_id": "float64", "drug": "string"},
    ):
        chunk = chunk[chunk["hadm_id"].isin(wanted)]
        if len(chunk):
            parts.append(chunk.drop_duplicates())
    rx = pd.concat(parts, ignore_index=True).drop_duplicates()
    rx["hadm_id"] = rx["hadm_id"].astype("int64")
    return rx.groupby("hadm_id").size().rename("n_distinct_drugs").reset_index()

if USE_PRESCRIPTIONS:
    drug_counts = cached("drug_counts", extract_drug_counts)
    cohort = cohort.merge(drug_counts, on="hadm_id", how="left")
    cohort["n_distinct_drugs"] = cohort["n_distinct_drugs"].fillna(0)
    rx_cols = ["n_distinct_drugs"]
    print(f"Polypharmacy: median {cohort['n_distinct_drugs'].median():.0f} distinct drugs, "
          f"90th pct {cohort['n_distinct_drugs'].quantile(0.9):.0f}")
else:

```

```

rx_cols = []
print("USE_PRESCRIPTIONS is False - skipping medication features.")

# -----
# 5.5 Assemble the modelling table
# -----
# The feature lists are declared explicitly rather than written as
# `X = cohort.drop(columns=["readmit_30d"])`, which would allow columns such as `days_to_next`
# into the model and produce leakage (an AUROC near 1.00).

NUMERIC_FEATURES = [
    # demographics & index stay
    "age", "los_days", "ed_los_hours", "came_via_ed",
    # ICU exposure
    "n_icu_stays", "icu_los_days", "icu_los_max",
    # comorbidity
    "charlson_score", "n_diagnoses",
    # prior utilisation (backward-looking only)
    "n_prior_admits_365d", "n_prior_unplanned_365d", "n_prior_admits_total",
    "days_since_prev_discharge", "is_first_admission",
] + list(CHARLSON.keys()) + lab_cols + rx_cols

CATEGORICAL_FEATURES = [
    "gender", "admission_type", "admission_location", "discharge_location",
    "insurance", "marital_status", "first_careunit", "last_careunit",
]

# `race` is deliberately *not* a model input by default. It stays in the dataframe because we
# need it in Section 11 to audit whether the model performs unevenly across groups. Section 11
# explains the reasoning; it is a judgement call, not a settled question.
if USE_RACE_AS_FEATURE:
    CATEGORICAL_FEATURES = CATEGORICAL_FEATURES + ["race"]

TARGET = "readmit_30d"
GROUP = "subject_id" # used to keep a patient's admissions together when splitting

# Columns that MUST NEVER reach the model. Listing them makes the intent explicit and gives
# us something to assert against.
FORBIDDEN = [
    "next_hadm_id", "next_admittime", "next_admission_type", "days_to_next",
    "dod", "days_to_death", "died_within_window", "deathtime", "hospital_expire_flag",
    "disctime", "admittime", "edregtime", "edouttime",
]

model_cols = NUMERIC_FEATURES + CATEGORICAL_FEATURES
missing = [c for c in model_cols if c not in cohort.columns]
assert not missing, f"These declared features are not in the cohort: {missing}"
assert not set(model_cols) & set(FORBIDDEN), "A forbidden column reached the feature list!"

extra_cols = [c for c in [TARGET, GROUP, "race", "anchor_year_group"] if c not in model_cols]
data = cohort[model_cols + extra_cols].copy()

print(f"Modelling table: {data.shape[0]:,} rows x {len(model_cols)} features "
      f"({len(NUMERIC_FEATURES)} numeric, {len(CATEGORICAL_FEATURES)} categorical)")
print(f"Positive class: {data[TARGET].mean():.2%}")
# The modelling table is ROW-LEVEL patient data, so it is saved in CACHE_DIR with the other
# intermediate files: useful for local debugging, and never to be shared or uploaded
# (keep the whole readmission_project/ folder on your own machine).
data.to_csv(CACHE_DIR / "modelling_table.csv", index=False)

# A feature dictionary instead of `data.head()`: what each input is, how complete it is, and
# a typical value - everything a reader needs, with no patient rows in the output.
def typical_value(col):
    s = data[col]
    if not s.notna().any():
        return "all missing"
    if col in NUMERIC_FEATURES:
        return f"median {s.median():.2f}"

```

```

return f"{s.nunique()} levels; most common: {s.mode(dropna=True).iloc[0]}"

feature_dictionary = pd.DataFrame({
    "kind": ["numeric"] * len(NUMERIC_FEATURES) + ["categorical"] * len(CATEGORICAL_FEATURES),
    "missing_%": (data[model_cols].isna().mean() * 100).round(1).to_numpy(),
    "typical_value": [typical_value(c) for c in model_cols],
}, index=pd.Index(model_cols, name="feature"))
display(feature_dictionary)

```

6. Exploratory analysis

Before modelling, three questions should be answered:

1. **How imbalanced is the outcome?** It determines which metrics are meaningful.
2. **What is missing, and is the missingness random?** In an EHR, a lab that was not ordered reflects a clinical decision rather than chance.
3. **Do the obvious risk factors move in the expected direction?** If prior admissions do not predict readmission, the pipeline should be checked for errors.

```

# -----
# 6.1 Missingness
# -----
miss = (data[model_cols].isna().mean() * 100).sort_values(ascending=False)
miss = miss[miss > 0]
print("Columns with missing values (%):")
display(miss.round(1).to_frame("percent_missing"))

if len(miss):
    fig, ax = plt.subplots(figsize=(7, max(2.5, 0.25 * len(miss))))
    ax.barh(miss.index[::-1], miss.values[::-1], color=BAR_COLOR)
    ax.set_xlabel("Percent missing")
    ax.set_title("Missing data by feature")
    ax.grid(axis="y", visible=False)
    save_fig(fig, "fig03_missingness")
    plt.show()

# -----
# 6.2 Does the outcome vary the way clinical intuition says it should?
# -----
def rate_by(col, bins=None, labels=None, top=None):
    """Readmission rate and group size for a categorical or binned numeric column."""
    s = data[col]
    if bins is not None:
        s = pd.cut(s, bins=bins, labels=labels, include_lowest=True)
    elif top is not None:
        keep = s.value_counts().head(top).index
        s = s.where(s.isin(keep), other="(other)")
    out = data.groupby(s, observed=True)[TARGET].agg(["mean", "size"])
    out.columns = ["readmit_rate", "n"]
    return out.sort_values("readmit_rate", ascending=False)

print("--- By number of prior unplanned admissions in the past year ---")
display(rate_by("n_prior_unplanned_365d",
    bins=[-0.1, 0, 1, 2, 5, 1000],
    labels=["0", "1", "2", "3-5", "6+"]).round(3))

print("--- By Charlson score band ---")
display(rate_by("charlson_score",
    bins=[-0.1, 0, 2, 4, 6, 100],
    labels=["0", "1-2", "3-4", "5-6", "7+"]).round(3))

```

```

print("--- By discharge destination (top 8) ---")
display(rate_by("discharge_location", top=8).round(3))

print("--- By age band ---")
display(rate_by("age", bins=[17, 40, 55, 70, 85, 120],
               labels=["18-39", "40-54", "55-69", "70-84", "85+"] ).round(3))

# -----
# 6.3 One figure that summarises the signal
# -----
panels = [
    ("n_prior_unplanned_365d", [-0.1, 0, 1, 2, 5, 1000], ["0", "1", "2", "3-5", "6+"],
     "Prior unplanned admissions (1 yr)"),
    ("charlson_score", [-0.1, 0, 2, 4, 6, 100], ["0", "1-2", "3-4", "5-6", "7+"],
     "Charlson comorbidity score"),
    ("los_days", [0, 2, 5, 10, 20, 400], ["<2", "2-5", "5-10", "10-20", "20+"],
     "Length of stay (days)",
    ("age", [17, 40, 55, 70, 85, 120], ["18-39", "40-54", "55-69", "70-84", "85+"],
     "Age (years)"),
]
fig, axes = plt.subplots(2, 2, figsize=(10, 6.5))
baseline = data[TARGET].mean()
for ax, (col, bins, labels, title) in zip(axes.ravel(), panels):
    tab = data.groupby(pd.cut(data[col], bins=bins, labels=labels, include_lowest=True),
                      observed=True)[TARGET].mean()
    ax.bar(tab.index.astype(str), tab.values, color=BAR_COLOR)
    ax.axhline(baseline, label="cohort average", linewidth=1.2, **style(1))
    ax.set_title(title, fontsize=10)
    ax.set_ylabel("30-day readmission rate")
    ax.grid(axis="x", visible=False)
axes[0, 0].legend(fontsize=8, loc="upper left")
fig.suptitle("Readmission rate by candidate risk factor", fontsize=12)
fig.tight_layout()
save_fig(fig, "fig04_risk_factors")
plt.show()

```

7. Splitting the data by patient

A patient can contribute several hospitalisations. A random row-wise split places some of a patient's admissions in the training set and others in the test set. The model can then learn patient-specific characteristics (for example, baseline creatinine or usual length of stay) and is rewarded for recognising them at test time. The resulting AUROC is inflated by an amount that cannot be estimated afterwards.

The fix is a **group-aware split**: every row belonging to one `subject_id` goes to exactly one side. `StratifiedGroupKFold` does this while also keeping the outcome rate roughly equal across folds.

Row-wise split (incorrect)	Group-wise split (correct)
patient A: [train][test]	patient A: [train][train]
patient B: [test][train]	patient B: [test][test]
^ leak	^ no overlap

We also build a **temporal split** (Section 12): train on the earliest `anchor_year_group`s and test on the latest. A random split answers "does this work on patients like these?"; a temporal split answers "does this work on future patients?", which is the more relevant question for deployment.

```

# -----
# 7.1 Group-aware train / test split
# -----
from sklearn.model_selection import GroupShuffleSplit, StratifiedGroupKFold

splitter = GroupShuffleSplit(n_splits=1, test_size=TEST_SIZE, random_state=RANDOM_SEED)
train_idx, test_idx = next(splitter.split(data, data[TARGET], groups=data[GROUP]))

train = data.iloc[train_idx].reset_index(drop=True)
test = data.iloc[test_idx].reset_index(drop=True)

X_train, y_train, g_train = train[model_cols], train[TARGET], train[GROUP]
X_test, y_test = test[model_cols], test[TARGET]

print(f"Train: {len(train):>7,} admissions  {train[GROUP].nunique():>7,} patients  "
      f"{y_train.mean():.2%} positive")
print(f"Test : {len(test):>7,} admissions  {test[GROUP].nunique():>7,} patients  "
      f"{y_test.mean():.2%} positive")

# Verify the split: no patient may appear in both sets.
overlap = set(train[GROUP]) & set(test[GROUP])
assert not overlap, f"{len(overlap)} patients appear in both splits; the split is invalid"
print(f"\nPatient overlap between train and test: {len(overlap)} OK")

# For reference: how many patients would a row-wise split have shared?
naive_train, naive_test = np.split(
    np.random.default_rng(0).permutation(len(data)), [int(0.8 * len(data))])
naive_overlap = (set(data.iloc[naive_train][GROUP]) & set(data.iloc[naive_test][GROUP]))
print(f"A naive row-wise split would have shared {len(naive_overlap):,} patients "
      f"({len(naive_overlap) / data[GROUP].nunique():.1%} of all patients) across the split.")

```

```

# -----
# 7.2 Preprocessing pipeline
# -----
# Everything that *learns* from data (the median used for imputation, the scaler's mean and
# standard deviation, the set of one-hot categories) must be fitted on the TRAINING data only.
# Wrapping all steps in an sklearn Pipeline is what makes that
# guarantee automatic when you cross-validate.
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

def make_ohe():
    """OneHotEncoder with a cross-version-compatible sparse argument."""
    try:
        # scikit-learn >= 1.2
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False, min_frequency=20)
    except TypeError:
        # older scikit-learn
        return OneHotEncoder(handle_unknown="ignore", sparse=False)

def build_preprocessor(scale=True):
    """Numeric: median-impute (+ optionally scale). Categorical: mode-impute + one-hot.

    `scale=False` is used for the tree ensembles, which do not care about feature scale.
    """
    numeric_steps = [("impute", SimpleImputer(strategy="median", add_indicator=True))]
    if scale:
        numeric_steps.append(("scale", StandardScaler()))

    return ColumnTransformer(
        transformers=[
            ("num", Pipeline(numeric_steps), NUMERIC_FEATURES),
            ("cat", Pipeline([
                ("impute", SimpleImputer(strategy="most_frequent")),
                ("onehot", make_ohe()),
            ]), CATEGORICAL_FEATURES),

```

```

    ],
    remainder="drop",
    verbose_feature_names_out=False,
)

# `add_indicator=True` above is a small but important choice: for every numeric column with
# missing values it appends a binary "was this missing?" column. In an EHR, the fact that a
# test was never ordered carries information, and this lets the model use it.

demo = build_preprocessor().fit(X_train)
print(f"Feature matrix after preprocessing: {demo.transform(X_train.head(5)).shape[1]} columns")
print("First 12 expanded feature names:")
print(list(demo.get_feature_names_out()[:12]))

```

8. Models

Four model families are compared; each represents a different approach.

Model	Why it is here	What to watch
Penalised logistic regression	The baseline. Coefficients are interpretable as odds ratios.	If more complex models do not outperform it, it is preferred.
Random forest	Bagged trees; captures interactions with almost no tuning.	Probabilities are poorly calibrated out of the box.
Histogram gradient boosting	Usually the strongest model on tabular EHR data.	Easy to overfit; watch the CV/test gap.
Multi-layer perceptron	Represents the deep-learning family; shows the performance of a small neural network on tabular data of this size.	Requires scaled inputs; seldom outperforms tree ensembles on tabular data.

Expected performance. On structured readmission data, published AUROCs are typically **0.65-0.72**; a value near 0.95 suggests leakage. A modest AUROC is expected, because readmission depends substantially on housing, transport, social support and access to follow-up care, none of which are recorded in MIMIC-IV.

```

# -----
# 8.1 Define the models
# -----
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.neural_network import MLPClassifier

models = {
    # L2-penalised logistic regression. We leave class_weight=None on purpose. The popular
    # alternative, class_weight="balanced", up-weights the rarer positive class: that barely
    # changes the *ranking* (so AUROC is similar), but it inflates every predicted probability,
    # i.e. it degrades calibration (Section 9.3). Keeping the model unweighted and
    # choosing a threshold that matches the care team's capacity (Section 9.4) is the cleaner
    # way to deal with class imbalance here.
    "Logistic regression": Pipeline([
        ("pre", build_preprocessor(scale=True)),
        ("clf", LogisticRegression(
            penalty="l2", C=1.0, max_iter=3000, solver="lbfgs",
            class_weight=None, random_state=RANDOM_SEED)),
    ]),

```

```

1),

# Random forest. min_samples_leaf is the main guard against memorising single patients.
"Random forest": Pipeline([
    ("pre", build_preprocessor(scale=False)),
    ("clf", RandomForestClassifier(
        n_estimators=400, min_samples_leaf=20, max_features="sqrt",
        n_jobs=-1, random_state=RANDOM_SEED)),
1),

# Histogram gradient boosting: fast, handles missing values natively, and a strong default.
"Gradient boosting": Pipeline([
    ("pre", build_preprocessor(scale=False)),
    ("clf", HistGradientBoostingClassifier(
        max_iter=400, learning_rate=0.06, max_leaf_nodes=31,
        min_samples_leaf=40, l2_regularization=1.0,
        early_stopping=True, validation_fraction=0.1,
        random_state=RANDOM_SEED)),
1),

# A small feed-forward network. Two hidden layers are sufficient for ~100 tabular features;
# early_stopping holds out 10% of the training data to decide when to stop.
"Neural net (MLP)": Pipeline([
    ("pre", build_preprocessor(scale=True)),
    ("clf", MLPClassifier(
        hidden_layer_sizes=(64, 32), activation="relu", alpha=1e-3,
        batch_size=512, learning_rate_init=1e-3, max_iter=80,
        early_stopping=True, n_iter_no_change=8, random_state=RANDOM_SEED)),
1),
}

# Optional: add a boosted-tree library if it happens to be installed.
try:
    from xgboost import XGBClassifier
    models["XGBoost (optional)"] = Pipeline([
        ("pre", build_preprocessor(scale=False)),
        ("clf", XGBClassifier(
            n_estimators=500, learning_rate=0.05, max_depth=5, subsample=0.8,
            colsample_bytree=0.8, reg_lambda=1.0, eval_metric="logloss",
            tree_method="hist", random_state=RANDOM_SEED)),
    ])
    print("xgboost found - added to the comparison.")
except ImportError:
    print("xgboost not installed - skipping (optional).")

print(f"\n{len(models)} models to compare.")

```

```

# -----
# 8.2 Cross-validate on the training set
# -----
# We use StratifiedGroupKFold so that (a) no patient spans folds and (b) each fold has a
# similar outcome rate. Scoring on BOTH AUROC and average precision (AUPRC) from the start
# ensures that model comparisons account for class imbalance.
from sklearn.model_selection import cross_validate

cv = StratifiedGroupKFold(n_splits=N_CV_FOLDS, shuffle=True, random_state=RANDOM_SEED)

cv_rows = []
for name, pipe in models.items():
    t0 = time.time()
    scores = cross_validate(
        pipe, X_train, y_train, groups=g_train, cv=cv,
        scoring=["roc_auc", "average_precision"],
        n_jobs=1, error_score="raise",
    )
    cv_rows.append({
        "model": name,
        "cv_aucroc": scores["test_roc_auc"].mean(),
        "cv_aucroc_sd": scores["test_roc_auc"].std(),
    })

```

```

"cv_auprc": scores["test_average_precision"].mean(),
"cv_auprc_sd": scores["test_average_precision"].std(),
"fit_seconds": time.time() - t0,
})
print(f"{name:<22} AUROC {cv_rows[-1]['cv_auroc']:.3f} "
      f"({cv_rows[-1]['cv_auroc_sd']:.3f}) "
      f"AUPRC {cv_rows[-1]['cv_auprc']:.3f} "
      f"({cv_rows[-1]['fit_seconds']:.0f}s)")

```

```

cv_results = pd.DataFrame(cv_rows).sort_values("cv_auroc", ascending=False)
display(cv_results.round(3))

```

```

# -----
# 8.3 Fit each model on the full training set and score the held-out test set
# -----
fitted, test_probs = {}, {}
for name, pipe in models.items():
    t0 = time.time()
    pipe.fit(X_train, y_train)
    fitted[name] = pipe
    # predict_proba returns [P(class 0), P(class 1)]; we want column 1.
    test_probs[name] = pipe.predict_proba(X_test)[: , 1]
    print(f"fitted {name:<22} in {time.time() - t0:5.1f}s")

```

9. Evaluation beyond AUROC

With roughly one admission in five readmitted (in the default 30-day setup), accuracy is uninformative: predicting that no patient is readmitted already achieves about 80%. Four measures are reported, each answering a different question.

Metric	Question it answers	Baseline to compare against
AUROC	For a randomly chosen readmitted and non-readmitted patient, how often does the model rank the readmitted patient higher?	0.5 (random ranking)
AUPRC (average precision)	Among flagged patients, what fraction are true readmissions? Sensitive to imbalance, unlike AUROC.	the prevalence (≈ 0.20 here)
Brier score	Are the <i>probabilities themselves</i> right, not just their order? Lower is better.	prevalence $\times (1 - \text{prevalence})$
Operating-point metrics	If the care team can follow up 10% of discharges, what fraction of readmissions is identified, and what fraction of follow-ups is unnecessary?	—

The last row is the most relevant to clinical use: in deployment, a model is applied through a threshold, a worklist and a fixed amount of staff time.

```

# -----
# 9.1 Metric helpers
# -----
from sklearn.metrics import (
    roc_auc_score, average_precision_score, brier_score_loss, roc_curve,
    precision_recall_curve, confusion_matrix, log_loss,
)
from sklearn.calibration import calibration_curve, CalibratedClassifierCV

ALERT_RATE = 0.10 # the fraction of discharges a care team can realistically follow up

def threshold_for_alert_rate(probs, rate=ALERT_RATE):

```

```

"""The probability cut-off that flags exactly `rate` of patients (the highest-risk ones)."""
return float(np.quantile(probs, 1 - rate))

def evaluate(y_true, probs, alert_rate=ALERT_RATE):
    """Return a dictionary of discrimination, calibration and operating-point metrics."""
    y_true = np.asarray(y_true)
    thr = threshold_for_alert_rate(probs, alert_rate)
    pred = (probs >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()

    sens = tp / (tp + fn) if (tp + fn) else np.nan      # recall / true positive rate
    spec = tn / (tn + fp) if (tn + fp) else np.nan
    ppv = tp / (tp + fp) if (tp + fp) else np.nan      # precision
    prevalence = y_true.mean()

    return {
        "auroc": roc_auc_score(y_true, probs),
        "auprc": average_precision_score(y_true, probs),
        "brier": brier_score_loss(y_true, probs),
        "log_loss": log_loss(y_true, np.clip(probs, 1e-6, 1 - 1e-6)),
        "prevalence": prevalence,
        "threshold": thr,
        "alert_rate": pred.mean(),
        "sensitivity": sens,
        "specificity": spec,
        "ppv": ppv,
        # Number Needed to Evaluate: how many patients must be followed up to catch one
        # true readmission; an intuitive measure for a clinical audience.
        "nne": 1 / ppv if ppv and ppv > 0 else np.nan,
        # Lift: how much better than flagging patients at random.
        "lift": ppv / prevalence if prevalence > 0 else np.nan,
    }

results = pd.DataFrame(
    {name: evaluate(y_test, p) for name, p in test_probs.items()}
).T.sort_values("auroc", ascending=False)

print(f"Held-out test set: {len(y_test):,} admissions, "
      f"{y_test.mean():.2%} readmitted within {READMIT_WINDOW_DAYS} days")
print(f"Operating point: flag the top {ALERT_RATE:.0%} of predicted risk\n")
display(results.round(3))

# Add the CV results side by side - a large train/test gap signals overfitting.
comparison = cv_results.set_index("model")[["cv_auroc", "cv_auprc"]].join(
    results[["auroc", "auprc"]].rename(columns={"auroc": "test_auroc", "auprc": "test_auprc"})
)
comparison["auroc_gap"] = comparison["cv_auroc"] - comparison["test_auroc"]
print("\nCross-validation vs held-out test (a large positive gap = overfitting):")
display(comparison.round(3))

# Which model do we carry forward? We decide with CROSS-VALIDATION, not with the test set.
# Selecting the model by test AUROC would turn the test set into a validation set,
# and every number reported from here on would be optimistically biased. The test set is
# for measuring the choice, not for making it.
BEST_MODEL = cv_results.iloc[0]["model"]
print(f"\nModel carried forward (best cross-validated AUROC): {BEST_MODEL}")
if cv_results["cv_auroc"].isna().all():
    print("[!] Cross-validated AUROC is undefined (some folds contain no readmissions), so the "
          "first model in the list was carried forward.\n    This only happens on tiny extracts "
          "such as the 100-patient MIMIC-IV demo: treat the numbers below as a functional check only.")
if results.index[0] != BEST_MODEL:
    print(f"(The test set happens to rank {results.index[0]} first. We keep the model chosen "
          "before looking at the test set, which is the purpose of holding it out.)")

# -----
# 9.2 ROC and precision-recall curves
# -----

```

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))

# --- ROC -----
ax = axes[0]
for i, (name, probs) in enumerate(test_probs.items()):
    fpr, tpr, _ = roc_curve(y_test, probs)
    ax.plot(fpr, tpr, label=f"{name} ({results.loc[name, 'auroc']:.3f})",
            linewidth=1.6, **style(i))
ax.plot([0, 1], [0, 1], color="#bbbbbb", linewidth=1, linestyle="--")
ax.set_xlabel("False positive rate (1 - specificity)")
ax.set_ylabel("True positive rate (sensitivity)")
ax.set_title("ROC curves")
ax.legend(fontsize=8, loc="lower right")

# --- Precision-recall -----
# The dashed horizontal line is the prevalence: what precision you would get by flagging
# patients at random. On an imbalanced problem this is the appropriate reference line.
ax = axes[1]
for i, (name, probs) in enumerate(test_probs.items()):
    prec, rec, _ = precision_recall_curve(y_test, probs)
    ax.plot(rec, prec, label=f"{name} ({results.loc[name, 'auprc']:.3f})",
            linewidth=1.6, **style(i))
ax.axhline(y_test.mean(), color="#bbbbbb", linewidth=1, linestyle="--")
ax.text(0.55, y_test.mean() + 0.008, f"random baseline = {y_test.mean():.3f}", fontsize=8,
        color="#666666")
ax.set_xlabel("Recall (sensitivity)")
ax.set_ylabel("Precision (PPV)")
ax.set_title("Precision-recall curves")
ax.legend(fontsize=8, loc="upper right")

fig.tight_layout()
save_fig(fig, "fig05_roc_pr")
plt.show()

```

9.3 Calibration

A model can rank patients well and still produce inaccurate probabilities. If it predicts "35% risk" for a group of patients and only 12% of them are readmitted, decisions and cost estimates based on that number will be wrong.

A **calibration plot** bins patients by predicted risk and plots observed vs predicted. Perfect calibration is the 45° line. Tree ensembles and random forests are usually *under-confident in the tails*; neural nets are often *over-confident*.

The fix is **post-hoc recalibration**: fit a small monotone function (isotonic regression, or Platt scaling) that maps raw scores onto calibrated probabilities, using cross-validation inside the training set. It leaves the ranking — and therefore the AUROC — essentially untouched, and it improves the Brier score *when there is miscalibration to fix*. Logistic regression and gradient boosting trained on log-loss without class weights are often close to calibrated already; if recalibration changes little, that result should be reported.

```

# -----
# 9.3 Calibration curves, before and after recalibration
# -----
fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))

ax = axes[0]
for i, (name, probs) in enumerate(test_probs.items()):
    frac_pos, mean_pred = calibration_curve(y_test, probs, n_bins=10, strategy="quantile")
    ax.plot(mean_pred, frac_pos, marker="o", markersize=4,
            label=f"{name} (Brier {results.loc[name, 'brier']:.4f})", linewidth=1.4, **style(i))
ax.plot([0, 1], [0, 1], color="#bbbbbb", linestyle="--", linewidth=1)

```

```

ax.set_xlabel("Mean predicted probability")
ax.set_ylabel("Observed frequency")
ax.set_title("Calibration, as fitted")
ax.legend(fontsize=8, loc="upper left")
lim = max(0.35, min(1.0, float(np.percentile(np.concatenate(list(test_probs.values())), 99.5))))
ax.set_xlim(0, lim)
ax.set_ylim(0, lim)

# --- Recalibrate the best model -----
# CalibratedClassifierCV refits the base pipeline on internal CV folds and learns the
# mapping on the out-of-fold predictions, so the calibrator never sees its own training data.
try:
    # scikit-learn >= 1.2
    calibrated = CalibratedClassifierCV(estimator=models[BEST_MODEL], method="isotonic", cv=3)
except TypeError:
    # older scikit-learn
    calibrated = CalibratedClassifierCV(base_estimator=models[BEST_MODEL],
                                       method="isotonic", cv=3)

calibrated.fit(X_train, y_train)
prob_cal = calibrated.predict_proba(X_test)[: , 1]
cal_metrics = evaluate(y_test, prob_cal) # kept separate from `test_probs` on purpose:
# `results` was computed from `test_probs`, so adding a key here would break any cell
# above that looks a model name up in `results`.

ax = axes[1]
for i, (label, probs) in enumerate([(BEST_MODEL, test_probs[BEST_MODEL]),
                                   ("after isotonic recalibration", prob_cal)]):
    frac_pos, mean_pred = calibration_curve(y_test, probs, n_bins=10, strategy="quantile")
    ax.plot(mean_pred, frac_pos, marker="o", markersize=4, label=label, linewidth=1.6, **style(i))
ax.plot([0, 1], [0, 1], color="bbbbbb", linestyle="--", linewidth=1)
ax.set_xlabel("Mean predicted probability")
ax.set_ylabel("Observed frequency")
ax.set_title(f"Isotonic recalibration\n({BEST_MODEL})")
ax.legend(fontsize=8, loc="upper left")
ax.set_xlim(0, lim)
ax.set_ylim(0, lim)

fig.tight_layout()
save_fig(fig, "fig06_calibration")
plt.show()

print(f"{BEST_MODEL:<32} Brier {results.loc[BEST_MODEL, 'brier']:.4f} "
      f"AUROC {results.loc[BEST_MODEL, 'auroc']:.3f}")
print(f"{'+' isotonic recalibration':<32} Brier {cal_metrics['brier']:.4f} "
      f"AUROC {cal_metrics['auroc']:.3f}")
# The interpretation below is derived from the observed values.
brier_change = cal_metrics["brier"] - results.loc[BEST_MODEL, "brier"]
auroc_change = cal_metrics["auroc"] - results.loc[BEST_MODEL, "auroc"]
print(f"\nChange after recalibration: Brier {brier_change:+.4f}, AUROC {auroc_change:+.3f}")
if brier_change < -0.001 and abs(auroc_change) <= 0.01:
    print("Recalibration improved the probabilities (lower Brier) while leaving the ranking "
          "(AUROC) essentially unchanged: it fixes the numbers, not the order.")
elif brier_change < -0.001:
    print("Recalibration improved the probabilities (lower Brier), but AUROC moved too. Isotonic "
          "recalibration is a step function that ties patients together, and on a small test set "
          "those ties can shift the ranking.")
elif brier_change > 0.001:
    print("Recalibration made the test-set probabilities slightly WORSE (higher Brier): the "
          "calibrator fitted noise in its internal folds, which can happen when the model was "
          "already well calibrated. Compare both curves with the 45-degree line.")
else:
    print("Recalibration changed almost nothing, because this model was already close to "
          "calibrated - compare its curve with the 45-degree line in the left panel. That is a "
          "useful result: the raw probabilities can be reported as they are.")

```

9.4 From risk scores to decisions

The model outputs a probability, whereas a care team needs a worklist. A threshold connects the two, and choosing it is a *resource* decision rather than a statistical one:

- How many follow-up calls can the team make per day?
- What is the cost of a missed readmission compared with an unnecessary call?

The plot below sweeps the alert rate from 2% to 50% and shows the consequences of each choice. **Number Needed to Evaluate (NNE)** — how many patients you follow up to catch one true readmission — is usually the most intuitive measure for a clinical audience.

```
# -----
# 9.4 Operating-point sweep for the best (recalibrated) model
# -----
best_probs = probab_cal
rates = np.linspace(0.02, 0.50, 25)
sweep = pd.DataFrame([evaluate(y_test, best_probs, alert_rate=r) for r in rates])
sweep["target_alert_rate"] = rates

fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
ax = axes[0]
ax.plot(sweep["target_alert_rate"], sweep["sensitivity"], label="Sensitivity (caught)",
        linewidth=1.8, **style(0))
ax.plot(sweep["target_alert_rate"], sweep["ppv"], label="PPV (precision)",
        linewidth=1.8, **style(1))
ax.axvline(ALERT_RATE, color="#bbbbbb", linestyle=":", linewidth=1.2)
ax.set_xlabel("Fraction of discharges flagged")
ax.set_ylabel("Rate")
ax.set_title("Sensitivity and PPV by alert rate")
ax.legend(fontsize=9)

ax = axes[1]
ax.plot(sweep["target_alert_rate"], sweep["nne"], linewidth=1.8, **style(0))
ax.axvline(ALERT_RATE, color="#bbbbbb", linestyle=":", linewidth=1.2)
ax.set_xlabel("Fraction of discharges flagged")
ax.set_ylabel("Number needed to evaluate")
ax.set_title("Number needed to evaluate by alert rate")
fig.tight_layout()
save_fig(fig, "fig07_operating_point")
plt.show()

m = evaluate(y_test, best_probs, alert_rate=ALERT_RATE)
n_flagged = int(round(m["alert_rate"] * len(y_test)))
print(f"At a {ALERT_RATE:.0%} alert rate on {len(y_test):,} test discharges:")
print(f"  flag {n_flagged:,} patients (risk >= {m['threshold']:.3f})")
print(f"  catch {m['sensitivity']:.1%} of all readmissions")
print(f"  {m['ppv']:.1%} of flagged patients are readmitted (vs {m['prevalence']:.1%} at random)")
print(f"  lift = {m['lift']:.2f}x, NNE = {m['nne']:.1f} patients per readmission caught")

cm = confusion_matrix(y_test, (best_probs >= m["threshold"]).astype(int))
print("\nConfusion matrix at that threshold:")
display(pd.DataFrame(cm, index=["actual: no readmit", "actual: readmit"],
                     columns=["predicted: no alert", "predicted: alert"]))
```

10. Interpretability

Three complementary methods, in increasing order of computational cost:

1. **Logistic regression coefficients** → odds ratios. Global, exact for that model, and directly comparable with clinical knowledge.
2. **Permutation importance** → shuffle one column and see how much test AUROC falls. Model-agnostic, measured on held-out data, and it reflects the features the model relies on.

3. **Partial dependence** → the average predicted risk as one feature varies. Shows the *shape* of a relationship, not just its strength.

All three describe **association**, not causation. "Number of prior admissions is important" does not mean preventing admissions would lower risk; it means prior admissions are a marker for underlying clinical factors.

```
# -----
# 10.1 Logistic regression odds ratios
# -----
lr = fitted["Logistic regression"]
feat_names = lr.named_steps["pre"].get_feature_names_out()
coefs = lr.named_steps["clf"].coef_[0]

or_table = (
    pd.DataFrame({"feature": feat_names, "coef": coefs})
    .assign(odds_ratio=lambda d: np.exp(d["coef"]),
            abs_coef=lambda d: d["coef"].abs())
    .sort_values("abs_coef", ascending=False)
)
print("Because numeric features were standardised, each coefficient is the change in "
      "log-odds per ONE STANDARD DEVIATION of that feature (per category for one-hot columns).\n")
print("Top 15 by magnitude:")
display(or_table.head(15)[["feature", "coef", "odds_ratio"]].round(3))

top = or_table.head(18).sort_values("coef")
fig, ax = plt.subplots(figsize=(8, 6))
ax.barh(top["feature"], top["coef"], color=BAR_COLOR)
ax.axvline(0, color="black", linewidth=0.8)
ax.set_xlabel("Coefficient (log-odds); positive = higher readmission risk")
ax.set_title("Logistic regression: largest coefficients")
ax.grid(axis="y", visible=False)
save_fig(fig, "fig08_logreg_coefficients")
plt.show()
```

```
# -----
# 10.2 Permutation importance for the best model
# -----
# Because we pass the whole Pipeline, permutation happens on the ORIGINAL columns, so the
# result is one number per feature you actually engineered - not per one-hot dummy. That is
# far more readable.
#
# It is computed on a test subsample for speed; increase `n_sample`/`n_repeats` for more precise estimates.
from sklearn.inspection import permutation_importance

n_sample = min(8000, len(X_test))
sub = X_test.sample(n_sample, random_state=RANDOM_SEED)
sub_y = y_test.loc[sub.index]

t0 = time.time()
perm = permutation_importance(
    fitted[BEST_MODEL], sub, sub_y,
    scoring="roc_auc", n_repeats=5, random_state=RANDOM_SEED, n_jobs=-1,
)
print(f"permutation importance computed in {time.time() - t0:.0f}s on {n_sample:,} rows")

imp = (pd.DataFrame({"feature": sub.columns,
                    "drop_in_auroc": perm.importances_mean,
                    "sd": perm.importances_std})
    .sort_values("drop_in_auroc", ascending=False))
display(imp.head(15).round(4))

topk = imp.head(15).sort_values("drop_in_auroc")
fig, ax = plt.subplots(figsize=(8, 5.5))
ax.barh(topk["feature"], topk["drop_in_auroc"], xerr=topk["sd"],
        color=BAR_COLOR, error_kw={"ecolor": "#999999", "lw": 1})
```

```
ax.set_xlabel("Drop in test AUROC when the feature is shuffled")
ax.set_title(f"Permutation importance - {BEST_MODEL}")
ax.grid(axis="y", visible=False)
save_fig(fig, "fig09_permutation_importance")
plt.show()
```

```
# -----
# 10.3 Partial dependence: the shape of the relationship
# -----
from sklearn.inspection import PartialDependenceDisplay

pdp_features = [f for f in ["n_prior_unplanned_365d", "charlson_score", "los_days", "age"]
                 if f in X_train.columns]
pdp_sample = X_train.sample(min(4000, len(X_train)), random_state=RANDOM_SEED)

fig, axes = plt.subplots(1, len(pdp_features), figsize=(3.2 * len(pdp_features), 3.4))
PartialDependenceDisplay.from_estimator(
    fitted[BEST_MODEL], pdp_sample, features=pdp_features,
    ax=axes, line_kw={"color": "black", "linewidth": 1.8},
)
for ax in np.atleast_1d(axes):
    ax.set_ylabel("Predicted probability")
fig.suptitle(f"Partial dependence - {BEST_MODEL}", fontsize=12)
fig.tight_layout()
save_fig(fig, "fig10_partial_dependence")
plt.show()

print("Read these as: 'holding everything else at its observed distribution, how does average "
      "predicted risk change as this feature moves?' A flat line means the model does not use it. "
      "Non-monotone fluctuations at the extremes usually reflect sparse data rather than biology.")
```

```
# -----
# 10.4 Optional: SHAP values for per-patient explanations
# -----
# SHAP attributes a single prediction to individual features, which is what you would show
# at the bedside ("this patient is flagged mainly because of X and Y"). Install with
# `pip install shap`. Skipped silently if unavailable.
try:
    import shap

    pre = fitted[BEST_MODEL].named_steps["pre"]
    clf = fitted[BEST_MODEL].named_steps["clf"]
    Xs = pre.transform(X_test.sample(min(2000, len(X_test)), random_state=RANDOM_SEED))
    names = pre.get_feature_names_out()

    explainer = shap.Explainer(clf, Xs, feature_names=names)
    shap_values = explainer(Xs[:500])
    shap.plots.beeswarm(shap_values, max_display=15, show=False)
    plt.title(f"SHAP summary - {BEST_MODEL}")
    save_fig(plt.gcf(), "fig11_shap_summary")
    plt.show()
except ImportError:
    print("shap not installed - skipping (pip install shap to enable).")
except Exception as exc:
    print(f"SHAP step skipped: {type(exc).__name__}: {exc}")
```

11. Subgroup audit

A single AUROC hides the possibility that a model works well for one group of patients and poorly for another. A model deployed in a hospital is applied to everyone, so it needs to be checked on everyone.

We report, for each subgroup: outcome prevalence, AUROC, and — at the **single shared threshold** the model would actually be deployed with — the alert rate and sensitivity.

Three considerations for interpreting the table:

1. **Different prevalence is not by itself unfairness.** If one group genuinely has more readmissions, a well-calibrated model will flag more of them. This is expected behaviour.
2. **Different sensitivity at a shared threshold warrants attention.** If the model catches 60% of readmissions in one group and 35% in another, the second group systematically receives less of the benefit.
3. **Small groups have wide uncertainty.** An AUROC computed on 80 patients with 6 events is unreliable. We print group sizes next to every metric for exactly this reason, and leave AUROC blank for groups too small to estimate it. Groups with fewer than 11 admissions are never shown on their own: they are pooled into "(other)" (or withheld if even that is tiny), because such small cells are statistically unreliable and can reveal an individual patient's outcome.

Race. In MIMIC-IV, recorded race is an administratively collected variable that acts as a proxy for structural factors (insurance, neighbourhood, access to follow-up care, differential treatment) rather than a biological cause. Including it as a model input can improve accuracy while encoding those inequities into future decisions; that is the argument for `USE_RACE_AS_FEATURE = False`. The counter-argument — that a model blind to race cannot be checked for or corrected against disparities — is also valid, which is why we *audit* on race even though we do not *train* on it.

```
# -----
# 11.1 Subgroup metrics at a single shared operating threshold
# -----
shared_threshold = threshold_for_alert_rate(best_probs, ALERT_RATE)
MIN_GROUP_N, MIN_GROUP_EVENTS = 200, 20 # below either one, AUROC is too noisy to report
MIN_CELL = 11 # never show a group with fewer admissions than this
# (the widely used "no cells of 1-10" disclosure rule)

def subgroup_report(series, label, top=None):
    """Metrics per level of `series`, evaluated at the shared deployment threshold.

    Levels outside the `top` most common, and any level with fewer than MIN_CELL admissions,
    are pooled into "(other)". If that pooled group is itself smaller than MIN_CELL, it is
    left out of the table: a "group" of one or two patients would reveal their outcomes.
    """
    s = series.fillna("(missing)").astype(str) # missing is a level of its own, not "(other)"
    if top is not None:
        keep = s.value_counts().head(top).index
        s = s.where(s.isin(keep), other="(other)")
    counts = s.value_counts()
    s = s.where(~s.isin(counts[counts < MIN_CELL].index), other="(other)")

    # `test` was reset_index'd, so row position == index label; we can mask positionally,
    # which keeps `best_probs` (a plain numpy array) correctly aligned.
    y_arr = np.asarray(y_test)
    rows = []
    for level in sorted(s.unique().tolist()):
        mask = (s == level).to_numpy()
        pos = y_arr[mask]
        pr = best_probs[mask]
        flagged = (pr >= shared_threshold).astype(int)
        n, ev = len(pos), int(pos.sum())
        if n < MIN_CELL: # only a pooled "(other)" can still be this small
```

```

        continue
    too_small = n < MIN_GROUP_N or ev < MIN_GROUP_EVENTS
    rows.append({
        label: str(level),
        "n": n,
        "events": ev,
        "prevalence": pos.mean(),
        "auroc": np.nan if too_small else roc_auc_score(pos, pr),
        "alert_rate": flagged.mean(),
        "sensitivity": np.nan if ev == 0 else flagged[pos == 1].mean(),
        "ppv": np.nan if flagged.sum() == 0 else pos[flagged == 1].mean(),
    })
return pd.DataFrame(rows).sort_values("n", ascending=False).reset_index(drop=True)

test_race = test["race"]
age_band = pd.cut(test["age"], [17, 40, 55, 70, 85, 120],
                  labels=["18-39", "40-54", "55-69", "70-84", "85+"])

print(f"n = admissions in the test set. AUROC is left blank for groups with < {MIN_GROUP_N} "
      f"admissions or < {MIN_GROUP_EVENTS} events;\ngroups with < {MIN_CELL} admissions are pooled "
      f"into '(other)', or not shown at all if the pool is still that small.")
for series, label, top in [
    (test["gender"], "sex", None),
    (age_band.astype(str), "age band", None),
    (test["insurance"], "insurance", 5),
    (test_race, "race (audit only)", 6),
]:
    print(f"\n=== by {label} ===")
    display(subgroup_report(series, label, top=top).round(3))

# -----
# 11.2 Visualise the gap that matters: sensitivity at a shared threshold
# -----
rep = subgroup_report(test["insurance"], "insurance", top=5)
rep = rep[rep["n"] >= MIN_GROUP_N]

fig, ax = plt.subplots(figsize=(8, 3.8))
x = np.arange(len(rep))
ax.bar(x - 0.2, rep["prevalence"], width=0.4, color="#888888", label="Actual readmission rate")
ax.bar(x + 0.2, rep["sensitivity"], width=0.4, color="#333333",
       label=f"Sensitivity at shared {ALERT_RATE:.0%} threshold")
ax.set_xticks(x)
ax.set_xticklabels([f"{v}\n(n={n:,})" for v, n in zip(rep["insurance"], rep["n"])], fontsize=9)
ax.set_ylabel("Rate")
ax.set_title("Readmission rate and sensitivity by insurance group")
ax.legend(fontsize=9)
ax.grid(axis="x", visible=False)
save_fig(fig, "fig12_subgroup_audit")
plt.show()

```

12. Temporal validation

A random split assumes that future patients resemble past ones. In practice, coding changes (the ICD-9 to ICD-10 transition occurs within this dataset), care pathways change, patient populations change, and 2020 affected all of these.

`anchor_year_group` places each *patient* in a real 3-year calendar window (the one containing their anchor year). We train on patients from the earlier windows and test on patients from the latest one — the closest thing to a prospective evaluation that retrospective data allows. Because the grouping is per patient, this split is automatically patient-disjoint. It is also approximate: one patient's admissions can span several real years.

A temporal shift can hurt a model in two different ways, so check both:

1. **Discrimination can fall.** If the relationships the model learned have drifted, AUROC drops. A drop of a few hundredths is common.
2. **The base rate can move.** If readmissions become rarer (or commoner), PPV and number-needed-to-evaluate change and the probabilities become miscalibrated — even when AUROC holds steady. Comparing *mean predicted risk* with the *observed rate* shows this at a glance.

In either case, the temporal results are the more realistic estimate of performance after deployment and should be reported.

```
# -----
# 12.1 Train on earlier years, test on the most recent year group
# -----
year_groups = sorted(data["anchor_year_group"].dropna().unique())
print("Available year groups:")
display(data.groupby("anchor_year_group")[TARGET].agg(["size", "mean"]).round(3))

holdout_group = year_groups[-1] if len(year_groups) >= 2 else None
if holdout_group is not None:
    tr = data[data["anchor_year_group"] != holdout_group]
    te = data[data["anchor_year_group"] == holdout_group]

# Both sides need readmitted AND non-readmitted patients, or AUROC is undefined. This only
# arises on very small extracts (such as the 100-patient MIMIC-IV demo).
if holdout_group is None or tr[TARGET].nunique() < 2 or te[TARGET].nunique() < 2:
    print("Not enough year groups (or outcomes) for a temporal split - skipped.")
else:
    print(f"\nTrain on {year_groups[:-1]} -> {len(tr):,} admissions")
    print(f"Test on ['{holdout_group}'] -> {len(te):,} admissions")

# clone() is essential: `models[BEST_MODEL]` and `fitted[BEST_MODEL]` are the SAME
# object, so refitting it here would silently overwrite the model we evaluated above.
from sklearn.base import clone
temporal_pipe = clone(models[BEST_MODEL])
temporal_pipe.fit(tr[model_cols], tr[TARGET])
temporal_probs = temporal_pipe.predict_proba(te[model_cols])[:, 1]
temporal_metrics = evaluate(te[TARGET], temporal_probs)

# "Calibration-in-the-large": the average predicted risk vs the rate actually observed.
random_col = results.loc[BEST_MODEL].copy()
random_col["mean_predicted"] = float(np.mean(test_probs[BEST_MODEL]))
temporal_col = pd.Series(temporal_metrics)
temporal_col["mean_predicted"] = float(np.mean(temporal_probs))

side_by_side = pd.DataFrame({
    "random patient-wise split": random_col,
    f"temporal split (test = {holdout_group})": temporal_col,
}).loc[["auroc", "auprc", "brier", "prevalence", "mean_predicted", "sensitivity", "ppv", "nne"]]
display(side_by_side.round(3))

# Interpretation of the observed results, for both effects described above.
change = temporal_metrics["auroc"] - results.loc[BEST_MODEL, "auroc"]
print(f"\nAUC change under temporal validation: {change:+.3f}")
if change < -0.06:
    print("    A large drop: the model leans on something that is not stable over time. Check "
          "whether coding practice or care-unit naming changed between year groups.")
elif change < -0.01:
    print("    A modest drop, typical of temporal shift. Report this number rather than the "
          "random-split one.")
elif change > 0.02:
    print("    AUROC increased. This rarely indicates a better model: the newest patients may "
          "simply be easier to rank (a different case mix), and a small test set moves AUROC ")
```

```

        "by a few hundredths by chance.")
    else:
        print("    Discrimination held up: the model ranks the newest patients about as well.")

    obs, pred = temporal_metrics["prevalence"], temporal_col["mean_predicted"]
    print(f"2) Observed readmission rate {obs:.1%} vs mean predicted risk {pred:.1%} "
          f"(random split: {random_col['prevalence']:.1%} observed)")
    if pred > 1.2 * obs:
        print("    The base rate fell and the model now OVER-predicts: PPV and NNE get worse, and "
              "the probabilities would need recalibrating on recent data before deployment.")
    elif pred < obs / 1.2:
        print("    The base rate rose and the model now UNDER-predicts: it would miss risk unless "
              "recalibrated on recent data.")
    else:
        print("    Predicted and observed rates agree: calibration survived the shift.")

```

13. Save the model and results

The fitted pipeline, the metrics and sufficient metadata are saved so that a reader can determine exactly what produced each reported number — plus one summary file holding every number the slides need (13.3).

```

# -----
# 13.1 Persist model, metrics and run metadata
# -----
import joblib

joblib.dump(calibrated, MODEL_DIR / "readmission_model_calibrated.joblib")
joblib.dump(fitted[BEST_MODEL], MODEL_DIR / "readmission_model_raw.joblib")

run_record = {
    "created": pd.Timestamp.now().isoformat(timespec="seconds"),
    "data_source": "MIMIC-IV v3.1",
    "outcome": f"unplanned readmission within {READMIT_WINDOW_DAYS} days of discharge",
    "cohort": {
        "n_admissions": int(len(data)),
        "n_patients": int(data[GROUP].nunique()),
        "outcome_rate": float(data[TARGET].mean()),
    },
    "split": {"type": "GroupShuffleSplit by subject_id", "test_size": TEST_SIZE,
              "random_state": RANDOM_SEED},
    "features": {"numeric": NUMERIC_FEATURES, "categorical": CATEGORICAL_FEATURES,
                 "race_used_as_feature": USE_RACE_AS_FEATURE},
    "best_model": BEST_MODEL,
    "test_metrics": {k: (None if pd.isna(v) else float(v))
                     for k, v in results.loc[BEST_MODEL].items()},
    "calibrated_test_metrics": {k: (None if pd.isna(v) else float(v))
                                for k, v in cal_metrics.items()},
    "versions": {"python": sys.version.split()[0], "pandas": pd.__version__,
                 "numpy": np.__version__, "scikit_learn": sklearn.__version__},
}
with open(MODEL_DIR / "run_record.json", "w") as fh:
    json.dump(run_record, fh, indent=2)

results.round(4).to_csv(PROJECT_DIR / "test_metrics.csv")
cv_results.round(4).to_csv(PROJECT_DIR / "cv_metrics.csv", index=False)

print("Saved:")
for p in sorted(PROJECT_DIR.rglob("*")):
    if p.is_file() and "cache" not in p.parts:
        print(f"  {p.relative_to(PROJECT_DIR)}  ({p.stat().st_size / 1024:.0f} KB)")

```

```

# -----
# 13.2 Score a new patient (what deployment would actually look like)

```

```
# -----
# The saved pipeline takes a raw DataFrame with the original columns and handles imputation,
# encoding and scaling internally, which is the advantage of using a Pipeline rather than
# transforming the data by hand.
#
# We score a HYPOTHETICAL patient rather than a real row: every numeric input is set to the
# training-set median and every categorical input to its most common value. That shows the
# interface without printing any real person's record or outcome, which keeps this notebook's
# output aggregate-only - safe to share, in line with the Data Use Agreement.
loaded = joblib.load(MODEL_DIR / "readmission_model_calibrated.joblib")

def most_common(s):
    m = s.mode(dropna=True)
    return m.iloc[0] if len(m) else np.nan

typical = {c: X_train[c].median() for c in NUMERIC_FEATURES}
typical.update({c: most_common(X_train[c]) for c in CATEGORICAL_FEATURES})
example = pd.DataFrame([typical])[model_cols]

# A what-if: the same patient, but with three unplanned admissions in the past year.
what_if = example.copy()
what_if["n_prior_unplanned_365d"] = 3
for col in ("n_prior_admits_365d", "n_prior_admits_total"):
    what_if[col] = max(3, float(example[col].iloc[0]))
what_if["is_first_admission"] = 0

for label, row in [("Typical patient", example),
                  ("Same patient, 3 unplanned admissions last year", what_if)]:
    risk = loaded.predict_proba(row)[0, 1]
    decision = "FLAG for transitional-care follow-up" if risk >= shared_threshold else "no alert"
    print(f"{label:<48} risk {risk:6.1%} -> {decision}")

print(f"\nTest-set readmission rate:           {y_test.mean():.1%}")
print(f"Alert threshold ({ALERT_RATE:.0%} alert rate):           {shared_threshold:.1%}")
print("\n(A hypothetical patient does not validate the model. It demonstrates the interface, and "
      "shows the prediction responding when the prior-admission history changes.)")
```

```
# -----
# 13.3 One file with every number the slide deck needs
# -----
# Numbers copied onto slides by hand become outdated when the notebook is re-run with a different
# setting, so every headline number is collected directly from the variables above into
# results_summary.md (readable, paste-able) and results_summary.json (machine-readable).
# Everything in these two files is aggregate - counts, rates, metrics - so, unlike anything
# in CACHE_DIR, they are safe to share (e.g. when you build your slides).
from IPython.display import Markdown
```

```
FIGURE_CAPTIONS = {
    "fig01_cohort_flow": "Cohort construction flow (the CONSORT-style diagram)",
    "fig02_time_to_readmission": "Days from discharge to unplanned readmission",
    "fig03_missingness": "Missing data by feature",
    "fig04_risk_factors": "Readmission rate by candidate risk factor",
    "fig05_roc_pr": "ROC and precision-recall curves, all models",
    "fig06_calibration": "Calibration, as fitted and after isotonic recalibration",
    "fig07_operating_point": "Sensitivity, PPV and NNE across alert rates",
    "fig08_logreg_coefficients": "Logistic regression: strongest associations",
    "fig09_permutation_importance": "Permutation importance, model carried forward",
    "fig10_partial_dependence": "Partial dependence for four key features",
    "fig11_shap_summary": "SHAP summary (only if shap is installed)",
    "fig12_subgroup_audit": "Subgroup audit: prevalence vs sensitivity by insurance",
}
```

```
def md_table(df, index_label=None, digits=3):
    """Render a small DataFrame as a Markdown table (no extra packages needed)."""
    if index_label is not None:
        df = df.rename_axis(index_label).reset_index()
```

```

def fmt(v):
    if isinstance(v, (bool, np.bool_)):
        return str(v)
    if isinstance(v, (int, np.integer)):
        return f"{int(v):,}"
    if isinstance(v, (float, np.floating)):
        if pd.isna(v):
            return ""
        return f"{v:,.{digits}f}".rstrip("0").rstrip(".") if "." in f"{v:,.{digits}f}" else f"{v:,.{digits}f}"
    return str(v)

lines = ["| " + " | ".join(str(c) for c in df.columns) + " |", "| " + "---|" * df.shape[1]]
lines += ["| " + " | ".join(fmt(v) for v in row) + " |" for row in df.itertuples(index=False)]
return "\n".join(lines)

def to_jsonable(o):
    """Convert numpy / pandas scalars (and NaN) into plain JSON-friendly Python values."""
    if isinstance(o, dict):
        return {str(k): to_jsonable(v) for k, v in o.items()}
    if isinstance(o, (list, tuple)):
        return [to_jsonable(v) for v in o]
    if isinstance(o, (bool, np.bool_)):
        return bool(o)
    if isinstance(o, (int, np.integer)):
        return int(o)
    if isinstance(o, (float, np.floating)):
        return None if pd.isna(o) else round(float(o), 5)
    if o is None or o is pd.NA or o is pd.NaT:
        return None
    return o if isinstance(o, str) else str(o)

m_op = evaluate(y_test, best_probs, alert_rate=ALERT_RATE)
tn, fp, fn, tp = confusion_matrix(y_test, (best_probs >= m_op["threshold"]).astype(int),
                                  labels=[0, 1]).ravel()

subgroups = {
    "sex": subgroup_report(test["gender"], "sex"),
    "age band": subgroup_report(age_band.astype(str), "age band"),
    "insurance": subgroup_report(test["insurance"], "insurance", top=5),
    "race (audit only)": subgroup_report(test["race"], "race (audit only)", top=6),
}

temporal_done = "side_by_side" in globals()
figures = sorted(p.stem for p in FIG_DIR.glob("*.png"))

headline = {
    "Cohort": f"{len(data):,} admissions from {data[GROUP].nunique():,} patients",
    "Outcome rate": f"{data[TARGET].mean():.1%} ({int(data[TARGET].sum()):,} readmissions)",
    "Model carried forward (chosen by CV)": BEST_MODEL,
    "Test AUROC / AUPRC": f"{results.loc[BEST_MODEL, 'auroc']:.3f} / {results.loc[BEST_MODEL, 'auprc']:.3f}",
    "Brier: raw -> recalibrated": f"{results.loc[BEST_MODEL, 'brier']:.4f} -> {cal_metrics['brier']:.4f}",
    f"At a {ALERT_RATE:.0%} alert rate": (
        f"flags {tp + fp:,} of {len(y_test):,}; catches {m_op['sensitivity']:.1%} of readmissions; "
        f"PPV {m_op['ppv']:.1%}; NNE {m_op['nne']:.1f}; lift {m_op['lift']:.2f}x"),
    "Temporal-split AUROC": (f"{side_by_side.loc['auroc'].iloc[1]:.3f} (test = {holdout_group}) vs "
        f"{side_by_side.loc['auroc'].iloc[0]:.3f} random split"
        if temporal_done else "not run"),
}

headline_md = md_table(pd.DataFrame({"value": headline}), index_label="quantity")

md = [
    "# Results summary: 30-day unplanned readmission after an ICU stay (MIMIC-IV v3.1)", "",
    f"_Generated {pd.Timestamp.now():%Y-%m-%d %H:%M} by the tutorial notebook. ",
    "Aggregate results only: no patient-level data.", "",
    "## Headline numbers", "", headline_md, "",
    "## Cohort flow", "", md_table(flow_df), "",
    "## Cross-validation (training set, patient-grouped folds)", "",

```

```

md_table(cv_results.drop(columns="fit_seconds")), "",
"## Held-out test set", "",
md_table(results[["auroc", "auprc", "brier", "sensitivity", "ppv", "nne", "lift"]], index_label="model"),
"",
f"## Operating point: flag the top {ALERT_RATE:.0%} (model carried forward, recalibrated)", "",
md_table(pd.DataFrame({"predicted: no alert": [tn, fn], "predicted: alert": [fp, tp]},
                      index=["actual: no readmit", "actual: readmit"]), index_label=""), "",
"## What drives the predictions", "",
"Permutation importance (drop in test AUROC when the feature is shuffled):", "",
md_table(imp.head(10), digits=4), "",
"Logistic regression, largest coefficients (per SD for numeric features):", "",
md_table(or_table.head(10)[["feature", "coef", "odds_ratio"]]), "",
"## Subgroup audit (one shared threshold)", "",
f"n = test-set admissions. A blank AUROC means the group is too small to estimate it "
f"< {MIN_GROUP_N} admissions or < {MIN_GROUP_EVENTS} events); groups with < {MIN_CELL} "
f"admissions are pooled into '(other)' or withheld.", "",
]
for name, table in subgroups.items():
    md += [f"*By {name}*", "", md_table(table), ""]
md += ["## Temporal validation", "",
      md_table(side_by_side, index_label="metric") if temporal_done else "Not run.", "",
      "## Figures (in readmission_project/figures/)", ""]
md += [f"- {f}.png - {FIGURE_CAPTIONS.get(f, '')}" for f in figures]

summary = {
    "setup": {
        "data_source": "MIMIC-IV v3.1",
        "outcome": f"unplanned readmission within {READMIT_WINDOW_DAYS} days of discharge",
        "population": f"adults (>= {MIN_AGE}) discharged alive after a hospitalisation with an ICU stay",
        "alert_rate": ALERT_RATE, "random_seed": RANDOM_SEED, "test_size": TEST_SIZE,
        "cv_folds": N_CV_FOLDS, "use_labs": USE_LABS, "use_prescriptions": USE_PRESCRIPTIONS,
        "race_used_as_feature": USE_RACE_AS_FEATURE,
        "n_features": {"numeric": len(NUMERIC_FEATURES), "categorical": len(CATEGORICAL_FEATURES)},
    },
    "headline": headline,
    "cohort_flow": flow_df.to_dict(orient="records"),
    "split": {"train_admissions": len(train), "train_patients": train[GROUP].nunique(),
             "test_admissions": len(test), "test_patients": test[GROUP].nunique(),
             "naive_rowwise_split_shared_patients": len(naive_overlap)},
    "cross_validation": cv_results.to_dict(orient="records"),
    "test_metrics": results.rename_axis("model").reset_index().to_dict(orient="records"),
    "model_carried_forward": BEST_MODEL,
    "calibration": {"brier_raw": results.loc[BEST_MODEL, "brier"], "brier_recalibrated":
cal_metrics["brier"]},
    "operating_point": {"m_op", "n_test": len(y_test), "n_flagged": tp + fp,
                       "confusion": {"tn": tn, "fp": fp, "fn": fn, "tp": tp}},
    "top_permutation_importance": imp.head(10).to_dict(orient="records"),
    "top_logreg_coefficients": or_table.head(10)[["feature", "coef", "odds_ratio"]].to_dict(orient="records"),
    "subgroups": {k: v.to_dict(orient="records") for k, v in subgroups.items()},
    "temporal": side_by_side.to_dict() if temporal_done else None,
    "figures": figures,
}

(PROJECT_DIR / "results_summary.md").write_text("\n".join(md) + "\n", encoding="utf-8")
with open(PROJECT_DIR / "results_summary.json", "w", encoding="utf-8") as fh:
    json.dump(to_jsonable(summary), fh, indent=2)
print(f" saved -> {rel(PROJECT_DIR / 'results_summary.md')}")
print(f" saved -> {rel(PROJECT_DIR / 'results_summary.json')}")
display(Markdown("### Headline results\n\n" + headline_md))

```

14. Summary, limitations and extensions

Summary of results

Cell 13.3 collected the headline numbers from this run and saved them, with every supporting table, to `readmission_project/results_summary.md` (and `.json`). Numbers for slides should be taken from that file rather than retyped, and the file regenerated whenever a setting changes. The six key results are: cohort size and outcome rate; the model carried forward; its test AUROC and AUPRC; the results at the 10% alert rate (share of readmissions identified, NNE); and the temporal-split AUROC, shown next to the random-split one.

Limitations

1. **Single centre, historical.** One Boston academic medical centre, 2008–2022. The results may not generalise to other hospitals.
2. **Readmissions elsewhere are invisible.** A patient readmitted to a different hospital is labelled a non-event. The measured rate is a lower bound.
3. **Death is handled by exclusion.** The model estimates risk *for patients who survive the window*, which is not the same as risk for everyone at discharge.
4. **Billing codes are an imperfect measure of disease.** ICD codes are generated for reimbursement. The Charlson score built from them measures *coded* comorbidity, which correlates with, but is not identical to, disease burden.
5. **Important predictors are unavailable.** Housing stability, transport, caregiver support, health literacy, whether a follow-up appointment was actually booked. Their absence is a large part of why AUROC plateaus around 0.7.
6. **The model is a triage aid.** It ranks patients for a limited resource. It must not be used to delay or deny discharge, and it has not been prospectively validated.
7. **Performance is not uniform across subgroups** (Section 11). Report those numbers alongside the headline ones, never instead of them.

Possible extensions

- **Competing-risks survival model** (Fine-Gray) instead of excluding deaths.
 - **Add the first-24h ICU physiology** from `chartevents` using the same streaming pattern as Section 5.3 — vitals, ventilation, vasopressors.
 - **Add clinical text.** `MIMIC-IV-Note` is a separate PhysioNet module containing discharge summaries; a fine-tuned clinical transformer or an LLM-derived embedding on top of these structured features would constitute a multimodal-fusion project.
 - **Compare against a published score** such as LACE or HOSPITAL and report the difference.
 - **Reporting.** Report the study according to TRIPOD+AI, the reporting guideline for clinical prediction models.
-

15. Exercises

The following exercises consolidate understanding of the pipeline. Each is a small, self-contained change with a predictable direction of effect; an unexpected result indicates an area worth investigating.

1. **Change the horizon.** Set `READMIT_WINDOW_DAYS = 7`, then `90`. Which horizon is easier to predict, and why?
2. **Introduce leakage deliberately.** Add `days_to_next` to `NUMERIC_FEATURES` and re-run Section 9. AUROC will rise towards 1.0, which illustrates how leakage appears in practice and why implausibly high performance warrants investigation.
3. **Use a row-wise split.** Replace `GroupShuffleSplit` with a plain `train_test_split` and measure how much the AUROC inflates. Report the difference.
4. **Relax the label.** Count *any* unplanned admission within 30 days, not just the next one. How much does the outcome rate move?
5. **Make the elective decision explicit.** Set `UNPLANNED_TYPES = observed_types` (count everything). Does performance go up or down, and is the resulting model more or less useful?
6. **Tune something.** Run a small `RandomizedSearchCV` over the gradient-boosting hyperparameters using `StratifiedGroupKFold`. How much does tuning improve AUROC compared with the difference between logistic regression and boosting?
7. **Add a table.** Bring in `services.csv.gz` (the clinical service that cared for the patient) or `transfers.csv.gz` (count of ward moves) and assess whether either improves performance.
8. **Subgroup threshold.** Section 11 uses one shared threshold. What happens to overall sensitivity and to the subgroup gaps if you set a per-group threshold instead? Is that an improvement, and who should make that decision?

Appendix A — Troubleshooting

Symptom	Cause	Fix
<code>FileNotFoundError:</code> <code>MIMIC_DIR</code> does not exist or <code>Could not find table</code> ...	<code>MIMIC_DIR</code> is not the folder containing <code>hosp/</code> and <code>icu/</code>	Fix it in your <code>MIMIC_DIR</code> variable, <code>mimic_path.txt</code> , or the placeholder in 1.2 (usually one level up or down); the printout in 1.3 shows what it found
Kernel dies while reading <code>labevents</code>	Chunk too large for available RAM	Lower <code>CHUNKSIZE</code> to 500_000
<code>MemoryError</code> on <code>pd.concat</code>	Too many kept rows	Aggregate inside the chunk loop instead of collecting all of them
Outcome rate is 0.00%	<code>UNPLANNED_TYPES</code> does not match the spellings in your release	Re-read the value counts in Section 3.5 and edit the set
AUROC $\approx$ 1.00	Leakage	Check <code>FORBIDDEN</code> ; print <code>model_cols</code> and read every name
<code>ValueError: Found unknown categories</code>	A category in test never appeared in train	Already handled by <code>handle_unknown="ignore"</code> ; if you replaced

Symptom	Cause	Fix
		the encoder, put it back
Columns named <code>age_x</code> , <code>age_y</code>	You re-ran a merge cell twice	Restart the kernel and run from the top — merges are not idempotent
<code>to_parquet</code> raises <code>ImportError</code>	<code>pyarrow</code> not installed	<code>pip install pyarrow</code> , or ignore — the code falls back to CSV

Appendix B — Reproducibility checklist

- ☐ Kernel → Restart & Run All completes with no errors
- ☐ `RANDOM_SEED` is used by every split, model and sampler
- ☐ Cohort flow table is saved (`cohort_flow.csv`)
- ☐ Package versions recorded (`run_record.json`)
- ☐ Figures regenerate into `figures/` with stable filenames
- ☐ No patient-level data leaves the machine; only counts, metrics and plots
- ☐ Outputs cleared before the notebook is shared (*Kernel → Restart & Clear Output*)
- ☐ `readmission_project/` (cache, models) and `mimic_path.txt` are never uploaded or shared
- ☐ Every reported number appears in a saved CSV or JSON, not only in the notebook output

Appendix C — Glossary

Term	Meaning
<code>subject_id</code> / <code>hadm_id</code> / <code>stay_id</code>	Patient / hospitalisation / ICU stay identifiers in MIMIC-IV
Index admission	The hospitalisation from which we predict
Prediction time	The moment after which no information may be used (here, <code>disctime</code>)
Target leakage	Using information unavailable at prediction time
Group split	Splitting so all rows of one patient stay on one side
AUROC	Probability a random positive is ranked above a random negative
AUPRC	Area under precision-recall; baseline is the prevalence
Calibration	Agreement between predicted probability and observed frequency
Brier score	Mean squared error of predicted probabilities
PPV / NNE	Precision / patients followed up per true case caught
Competing risk	An event (death) that prevents the outcome from occurring
Charlson index	Weighted comorbidity score derived from ICD codes

Appendix D — Sources

- Johnson AEW, Bulgarelli L, Shen L, et al. *MIMIC-IV, a freely accessible electronic health record dataset*. Scientific Data, 2023.
- Quan H, Sundararajan V, Halfon P, et al. *Coding algorithms for defining comorbidities in ICD-9-CM and ICD-10 administrative data*. Medical Care, 2005.
- Charlson ME, Pompei P, Ales KL, MacKenzie CR. *A new method of classifying prognostic comorbidity in longitudinal studies*. Journal of Chronic Diseases, 1987.
- Collins GS, Moons KGM, Dhiman P, et al. *TRIPOD+AI statement*. BMJ, 2024.
- MIMIC-IV v3.1 on PhysioNet (cite the dataset exactly as its page shows): <https://physionet.org/content/mimiciv/3.1/>
- MIMIC code repository: <https://github.com/MIT-LCP/mimic-code>